



计算机组成原理 第八章 CPU 的结构和功能



系统结构研究所

大 纲

- (一) CPU的功能和基本结构
- (二) 指令执行过程
- (三) 数据通路的功能和基本结构
- (四) 控制器的功能和工作原理
 - 1. 硬布线控制器
 - 2. 微程序控制器
- (五) 异常和中断机制
 - 1.异常和中断的基本概念
 - 2. 异常和中断的分类
 - 3. 异常和中断的检测与响应
- (六) 指令流水线
 - 1. 指令流水线的基本概念
 - 2. 指令流水线的基本实现
 - 3. 结构冒险、数据冒险和控制冒险的处理
 - 4. 超标量和动态流水线的基本概念

Contents

8.1

CPU 的结构

8.2

指令周期

8.3

指令流水

8.4

中断系统

8.1 CPU 的结构

一、CPU 的功能

1. 控制器的功能

取指令

指令控制

分析指令

操作控制

执行指令，发出各种操作命令

时间控制

控制程序输入及结果的输出

总线管理

处理中断

处理异常情况和特殊请求

数据加工

2. 运算器的功能

实现算术运算和逻辑运算

二、CPU 结构框图

8.1

1. CPU 与系统总线

指令控制

PC IR

操作控制

时间控制



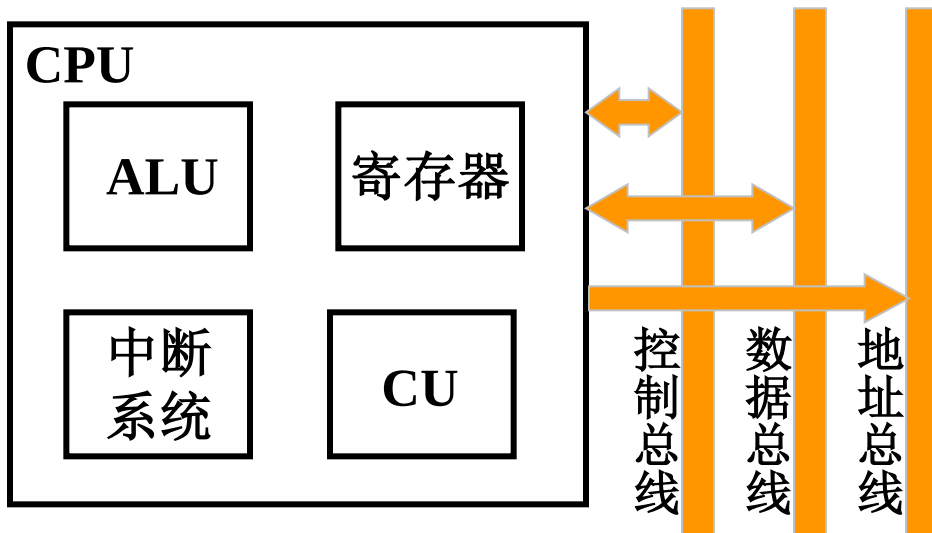
CU 时序电路

数据加工

ALU 寄存器

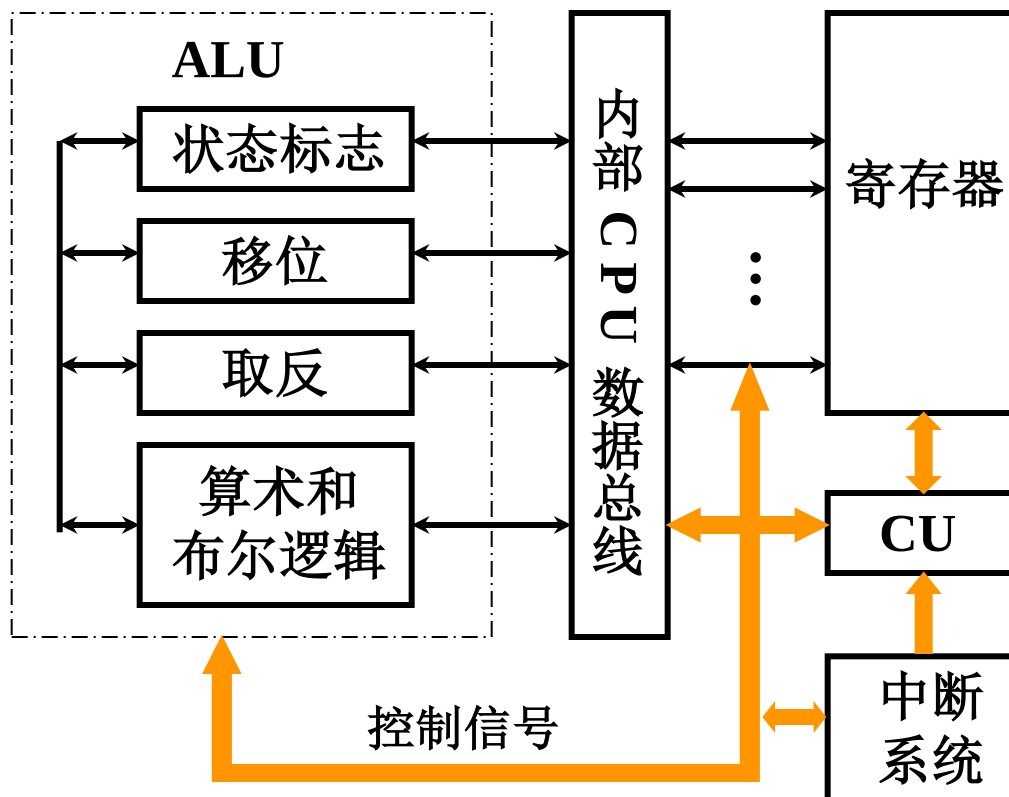
处理中断

中断系统



2. CPU 的内部结构

8.1



三、CPU 的寄存器

1. 用户可见寄存器

- (1) 通用寄存器 存放操作数
可作 某种寻址方式所需的 专用寄存器
- (2) 数据寄存器 存放操作数（满足各种数据类型）
两个寄存器拼接存放双倍字长数据
- (3) 地址寄存器 存放地址，其位数应满足最大的地址范围
用于特殊的寻址方式 段基值 栈指针
- (4) 条件码寄存器 存放条件码，可作程序分支的依据
如 正、负、零、溢出、进位等

(1) 控制寄存器

$PC \rightarrow MAR \rightarrow M \rightarrow MDR \rightarrow IR$

控制 CPU 操作

其中 **MAR**、**MDR**、**IR**

用户不可见

PC

用户可见

(2) 状态寄存器

状态寄存器

存放条件码

PSW 寄存器

存放程序状态字

1. CU 产生全部指令的微操作命令序列

组合逻辑设计

硬连线逻辑

微程序设计

存储逻辑

参见 第 4 篇

2. 中断系统

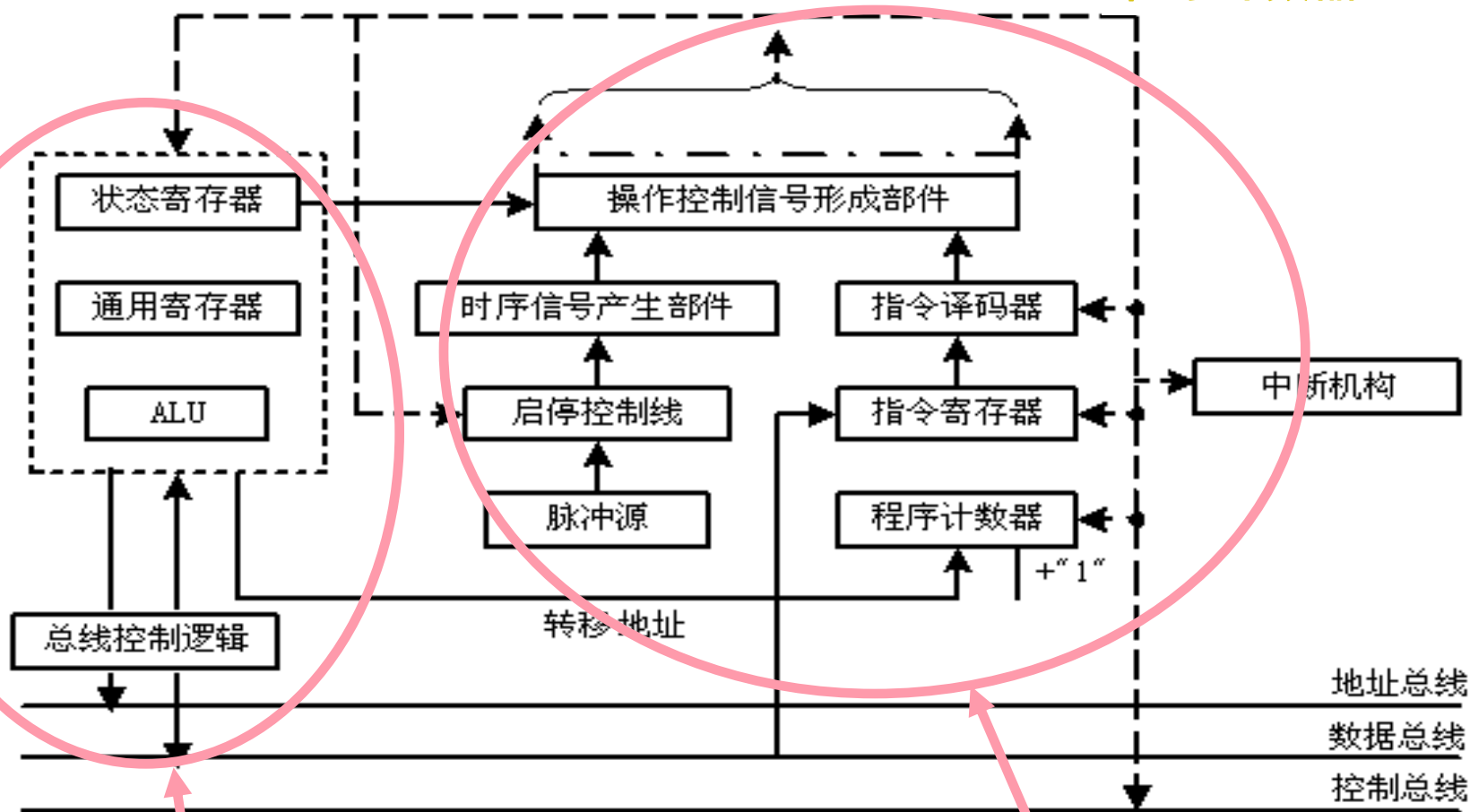
参见 8.4 节

五、ALU

参见 第 6 章

CPU基本组成原理图

指令寄存器---IR
程序计数器---PC



执行部件

控制部件

CPU 由 执行部件 和 控制部件 组成
CPU 包含 数据通路 和 控制器

控制器 由 指令译码器 和 控制信号形成部件 组成



- PC、IR
- 指令译码器ID：对操作进行译码，向控制器提供操作的特定信号。
- 时序发生器：产生各种时序信号
 - 脉冲源：通常由石英晶体振荡器构成。产生一定频率的脉冲信号作为整个机器的时钟脉冲，是机器周期和工作脉冲的基准信号。在机器刚加电时，产生总清信号(reset)
 - 启停线路：保证可靠地送出或封锁完整的时钟脉冲，控制时序信号的发生或停止，从而启动机器工作或停机。



- 时序控制信号形成部件：当机器启动后，在CLK时钟作用下，根据当前正在执行的指令的需要，产生相应的时序控制信号，并根据被控功能部件的反馈信号调整时序控制信号。
- 状态寄存器（PSR）：存放PSW,PSW表明了系统基本状态，是控制程序执行的重要依据。

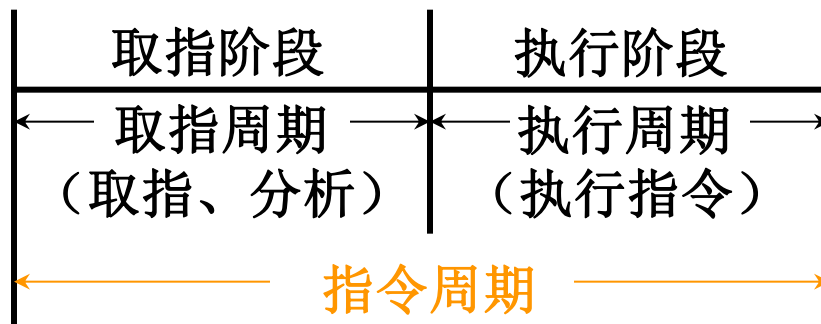
8.2 指令周期

一、指令周期的基本概念

1. 指令周期

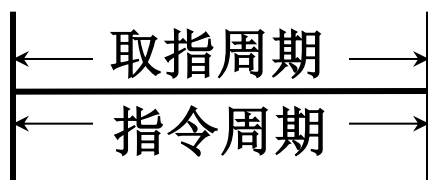
取出并执行一条指令所需的全部时间

完成一条指令 { 取指、分析 取指周期
 执行 执行周期

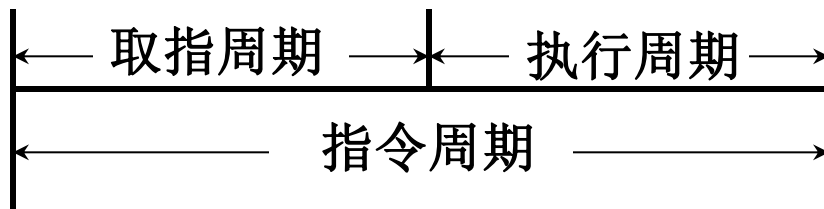


2. 每条指令的指令周期不同

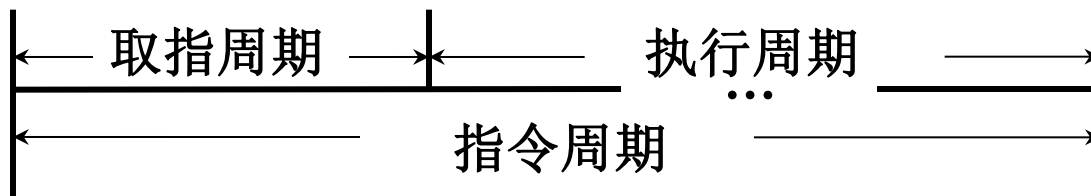
8.2



NOP



ADD mem

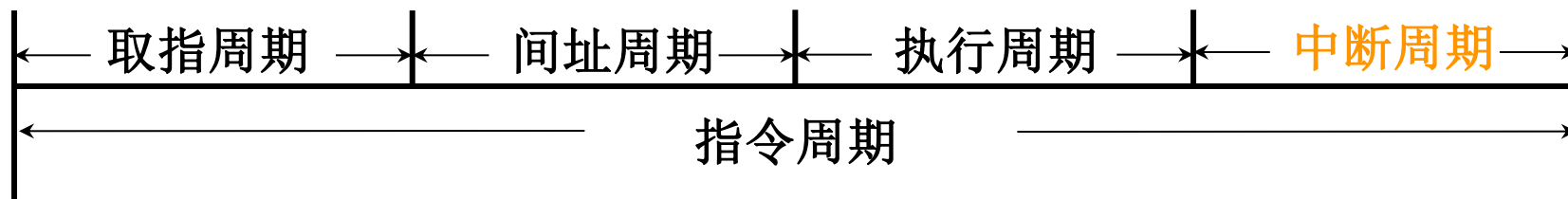


MUL mem

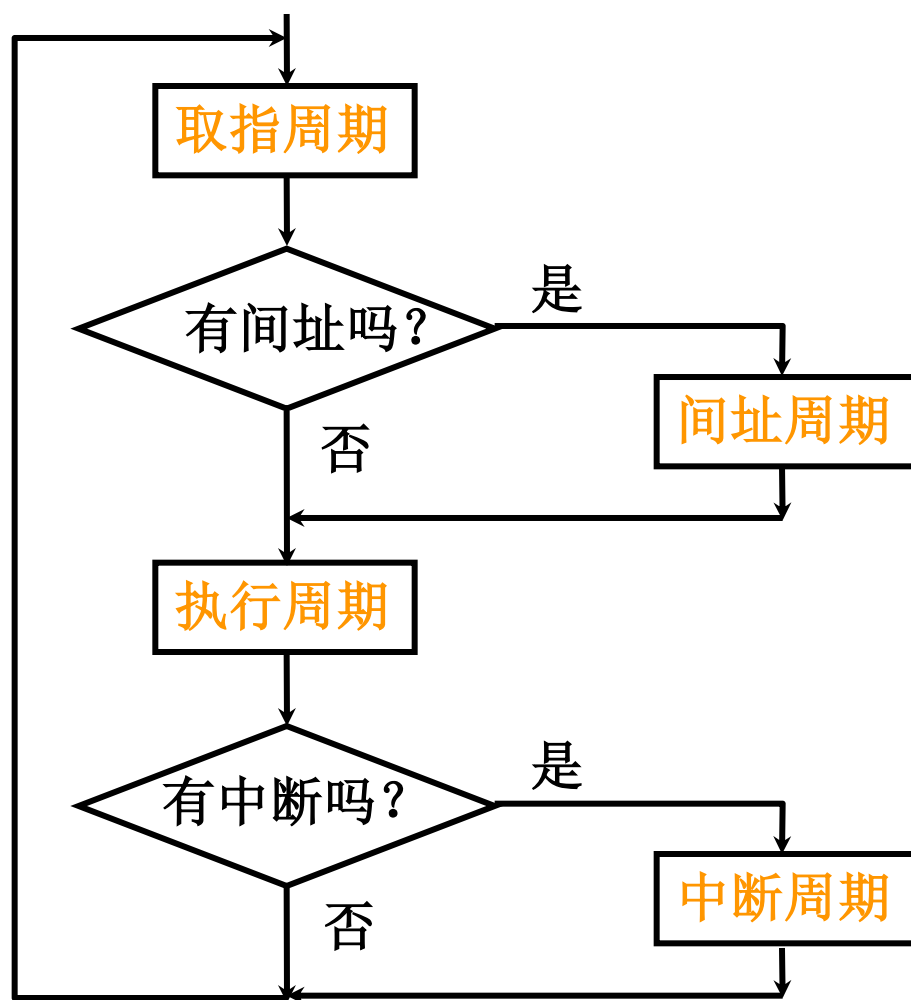
3. 具有间接寻址的指令周期



4. 带有中断周期的指令周期



5. 指令周期流程



6. CPU 工作周期的标志

CPU 访存有四种性质

取 指令

取指周期

取 地址

间址周期

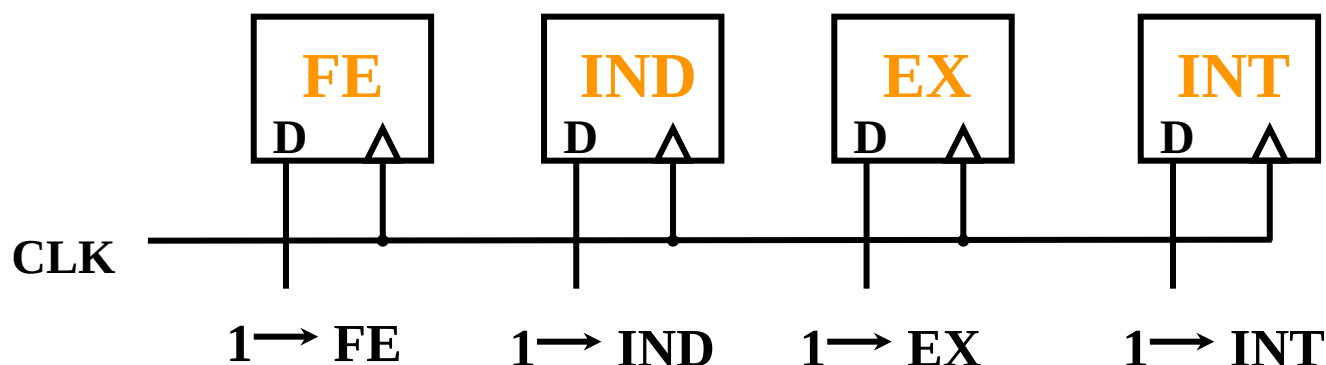
取 操作数

执行周期

存 程序断点

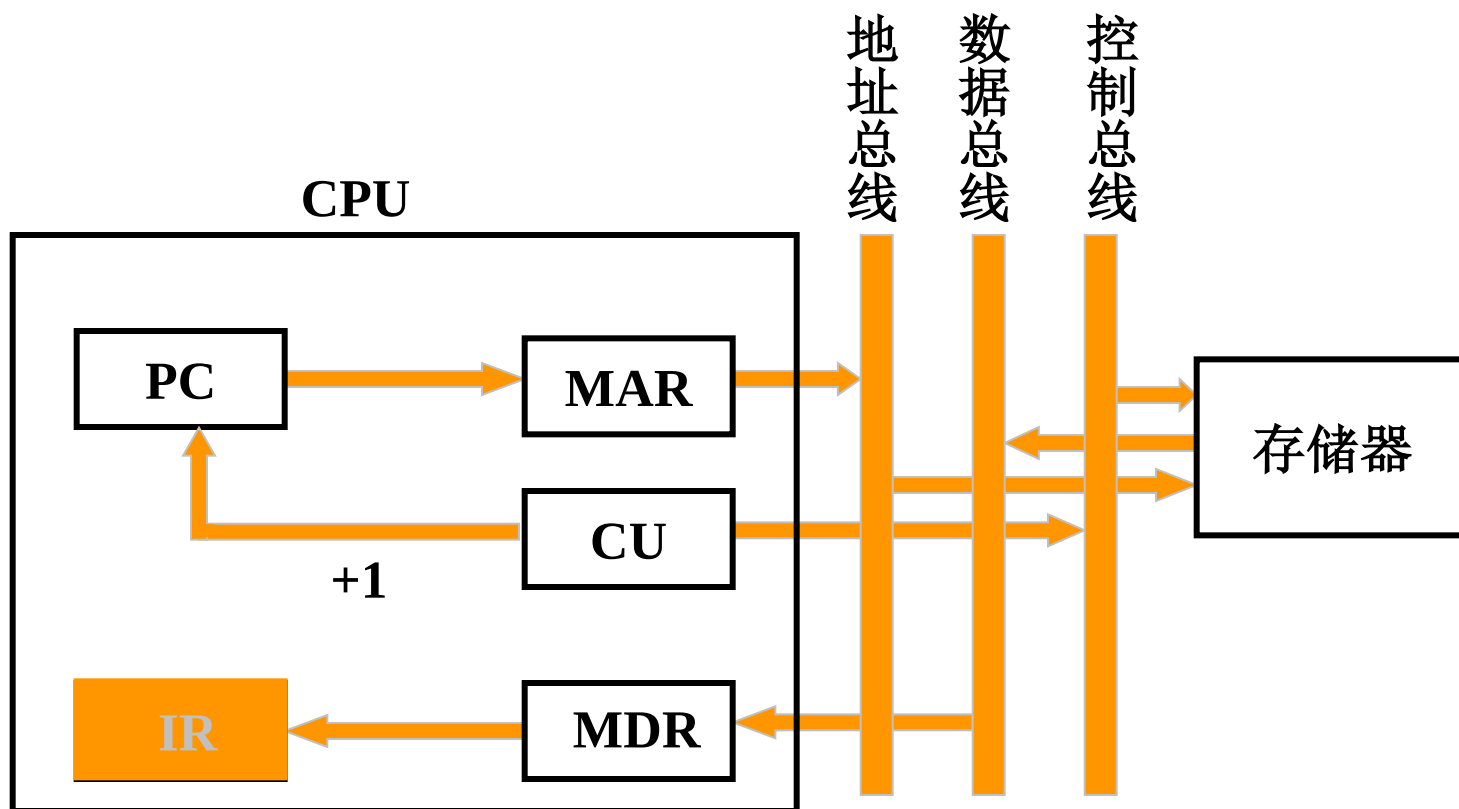
中断周期

CPU 的
4个工作周期



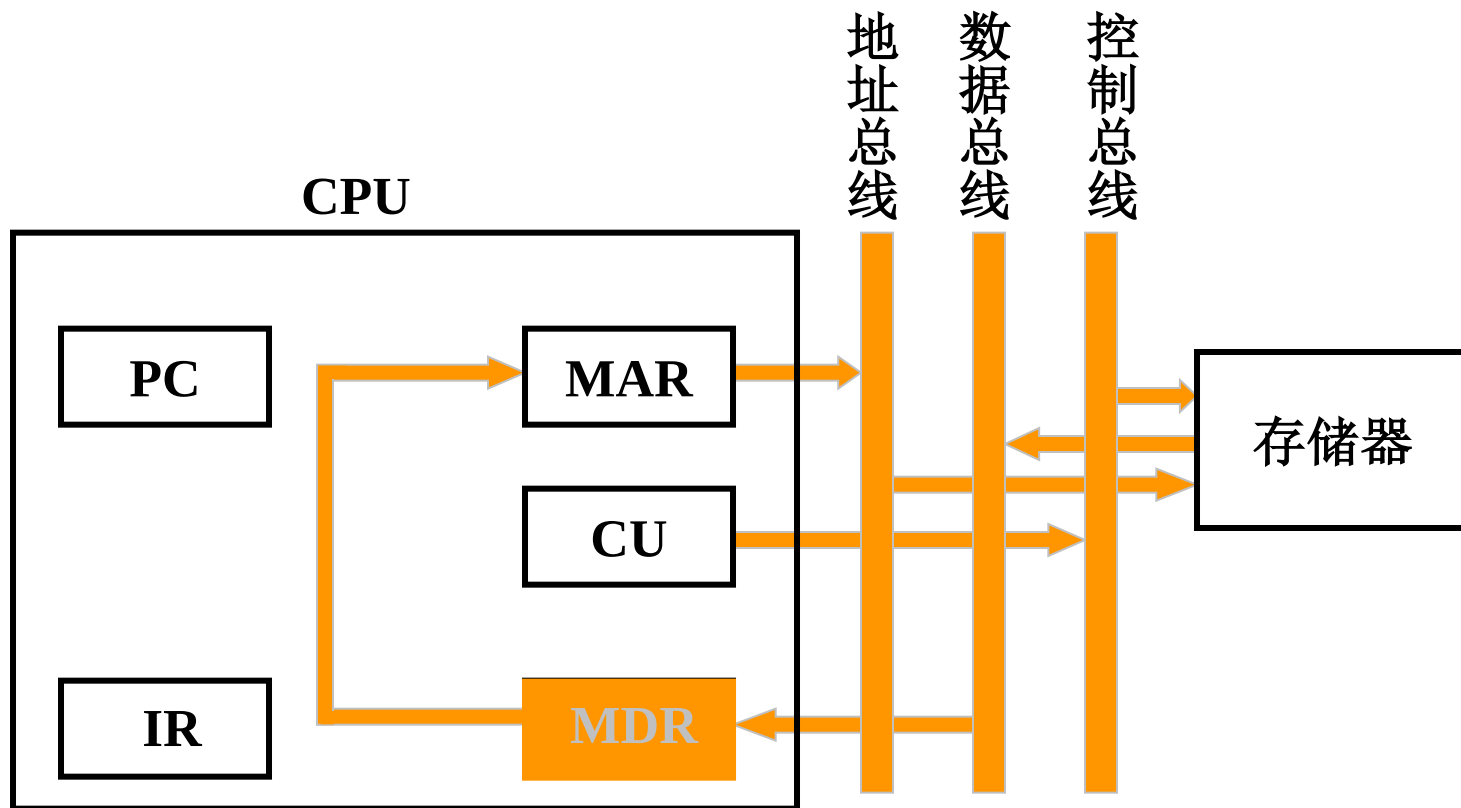
二、指令周期的数据流

1. 取指周期数据流



2. 间址周期数据流

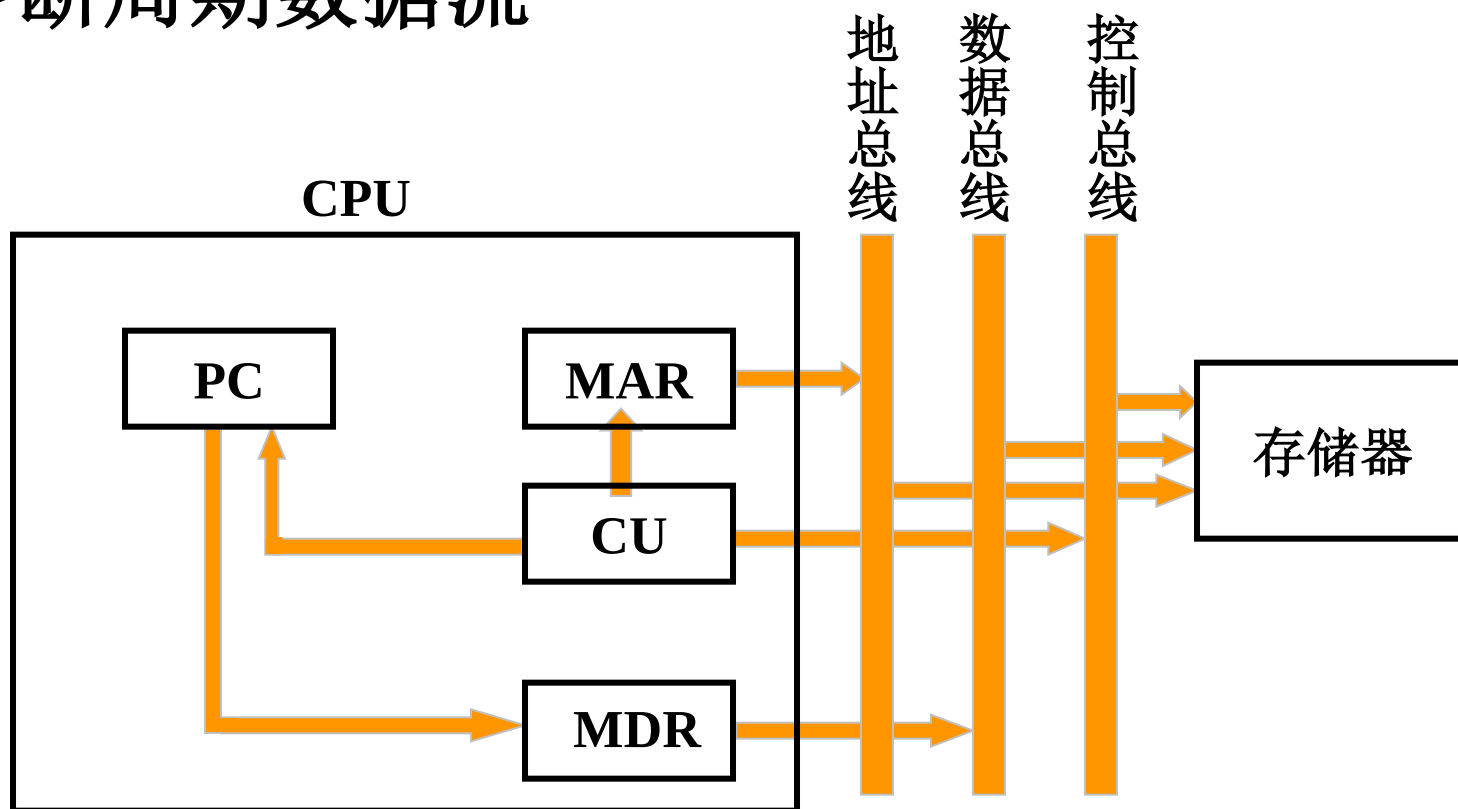
8.2



3. 执行周期数据流

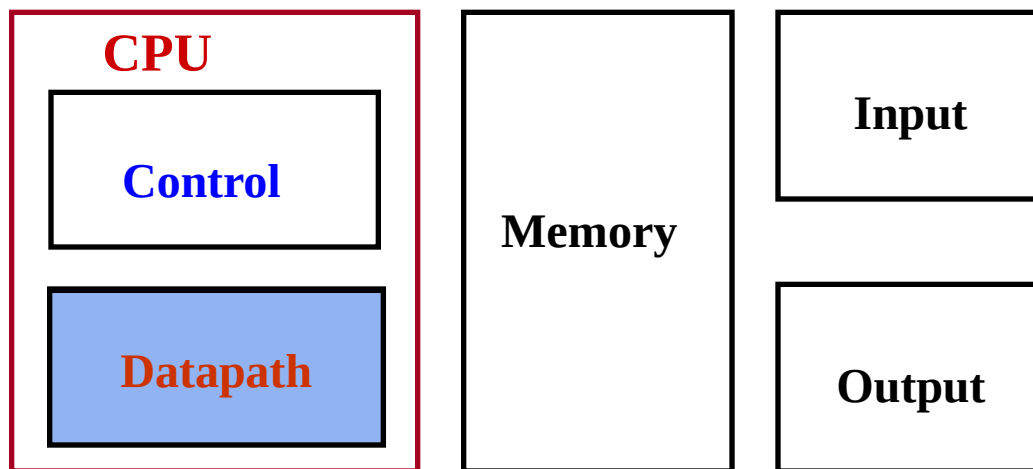
不同指令的执行周期数据流不同

4. 中断周期数据流



三、数据通路

❖ 计算机的五大组成部分：



❖ 什么是数据通路（DataPath）？

- 指令执行过程中，数据所经过的路径，包括路径中的部件。它是**指令的执行部件**。

❖ 控制器（Control）的功能是什么？

- 对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。能对执行部件发出控制信号，是**指令的控制部件**。

数据通路的基本结构

❖ 数据通路由两类部件组成

- 组合逻辑元件（也称操作元件）
- 存储元件（也称状态元件）

❖ 元件间的连接方式

- 总线连接方式
- 分散连接方式

❖ 数据通路如何构成？

- 由“操作元件”和“存储元件”通过总线方式或分散方式连接而成

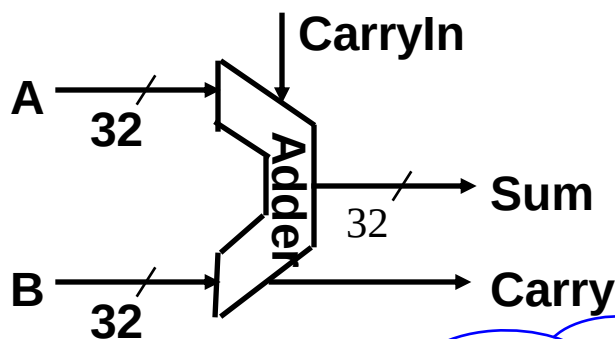
❖ 数据通路的功能是什么？

- 进行数据存储、处理、传送

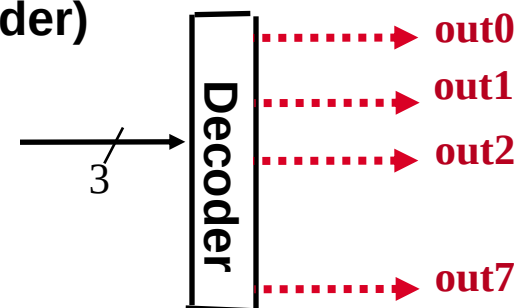
因此，数据通路是由操作元件和存储元件通过总线方式或分散方式连接而成的进行数据存储、处理、传送的路径。

.....► 控制信号

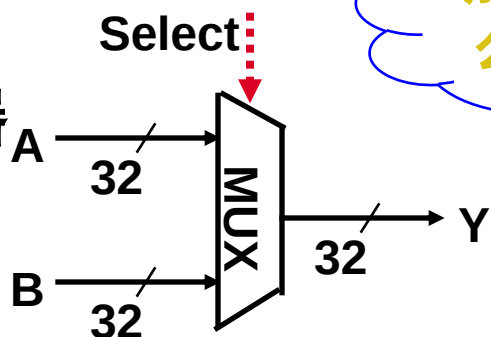
❖ 加法器 (Adder)



译码器 (Decoder)



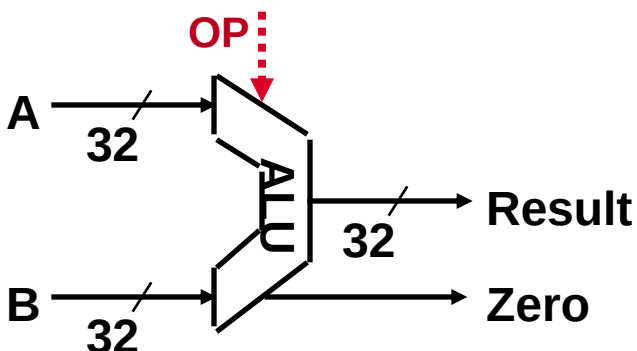
多路选择器 (MUX)



加法器需要什么控制信号?

何时要用到adder, ALU, MUX or Decoder?

算逻部件 (ALU)



组合逻辑元件的特点:

其输出只取决于当前的输入。即: 输入一样, 其输出也一样

定时: 所有输入到达后, 经过一定的逻辑门延时, 输出端改变, 并保持到下次改变, 不需要时钟信号来定时

状态元件：时序逻辑电路

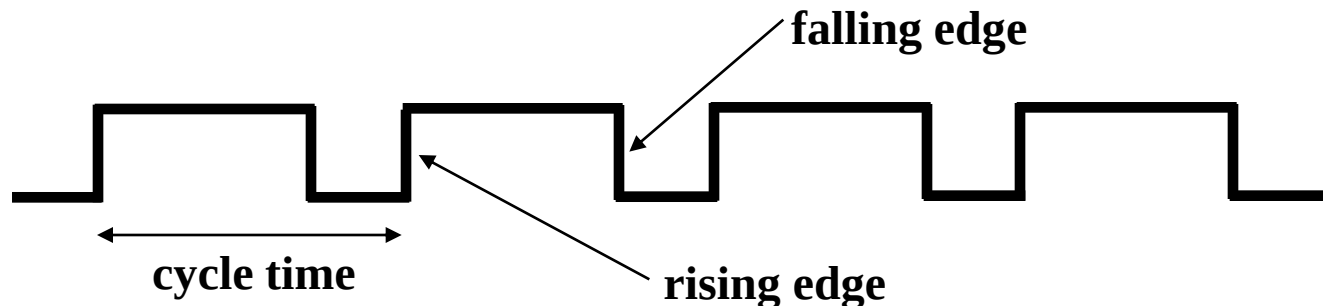
❖ 状态（存储）元件的特点：

- 具有存储功能，在**时钟控制**下输入被写到电路中，直到下个时钟到达
- 输入端状态由时钟决定何时被写入，输出端状态随时可以读出

❖ 定时方式：规定信号何时写入状态元件或何时从状态元件读出

■ 边沿触发（edge-triggered）方式：

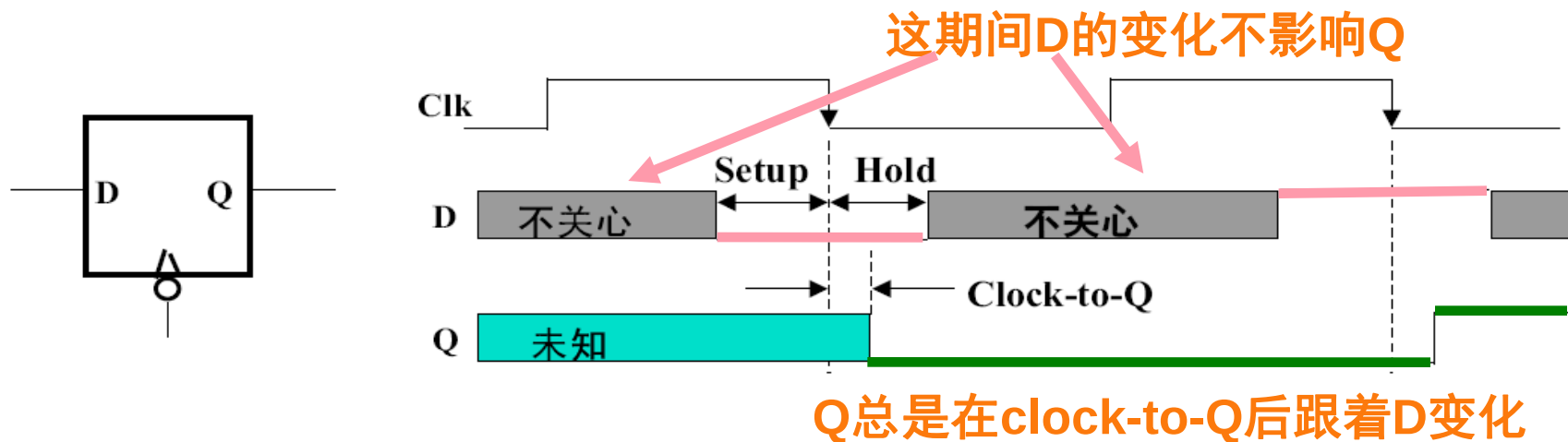
- 状态单元中的值只在时钟边沿改变。每个时钟周期改变一次。
 - 上升沿（rising edge）触发：在时钟正跳变时进行读/写。
 - 下降沿（falling edge）触发：在时钟负跳变时进行读/写。



❖ 最简单的状态单元（回顾：数字逻辑电路课程内容）：

- D触发器：一个时钟输入、一个状态输入、一个状态输出

存储元件中何时状态被改变？



- ° 建立时间（**Setup Time**）：在触发时钟边沿 **之前** 输入必须稳定
- ° 保持时间（**Hold Time**）：在触发时钟边沿 **之后** 输入必须保持
- ° **Clock-to-Q time: (Latch Prop - 锁存延迟)**
 - 在触发时钟边沿，输出并不能立即变化

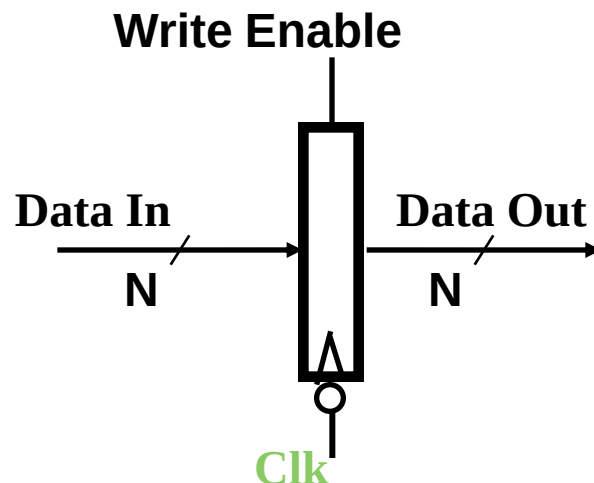
切记：状态单元的输入信息总是在一个时钟边沿到达后的“Clk-to-Q”时才被写入到单元中，此时的输出才反映新的状态值

数据通路中的状态元件有两种：寄存器(组) + 存储器

存储元件：寄存器和寄存器组

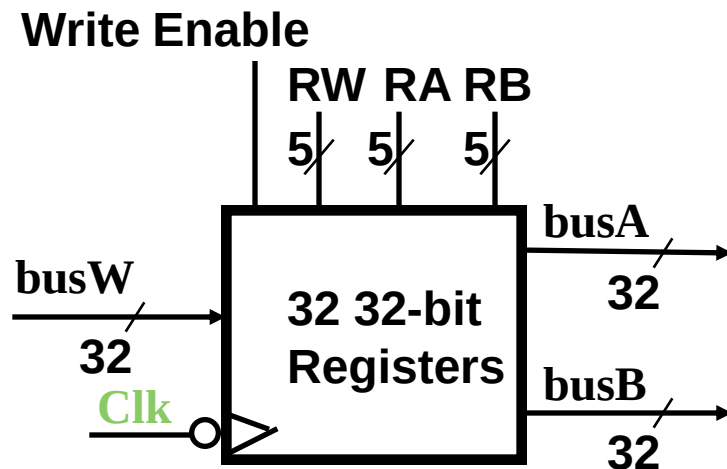
❖ 寄存器 (Register)

- 有一个写使能 (Write Enable-WE) 信号
 - 0: 时钟边沿到来时, 输出不变
 - 1: 时钟边沿到来时, 输出开始变为输入
- 若每个时钟边沿都写入, 则不需WE信号

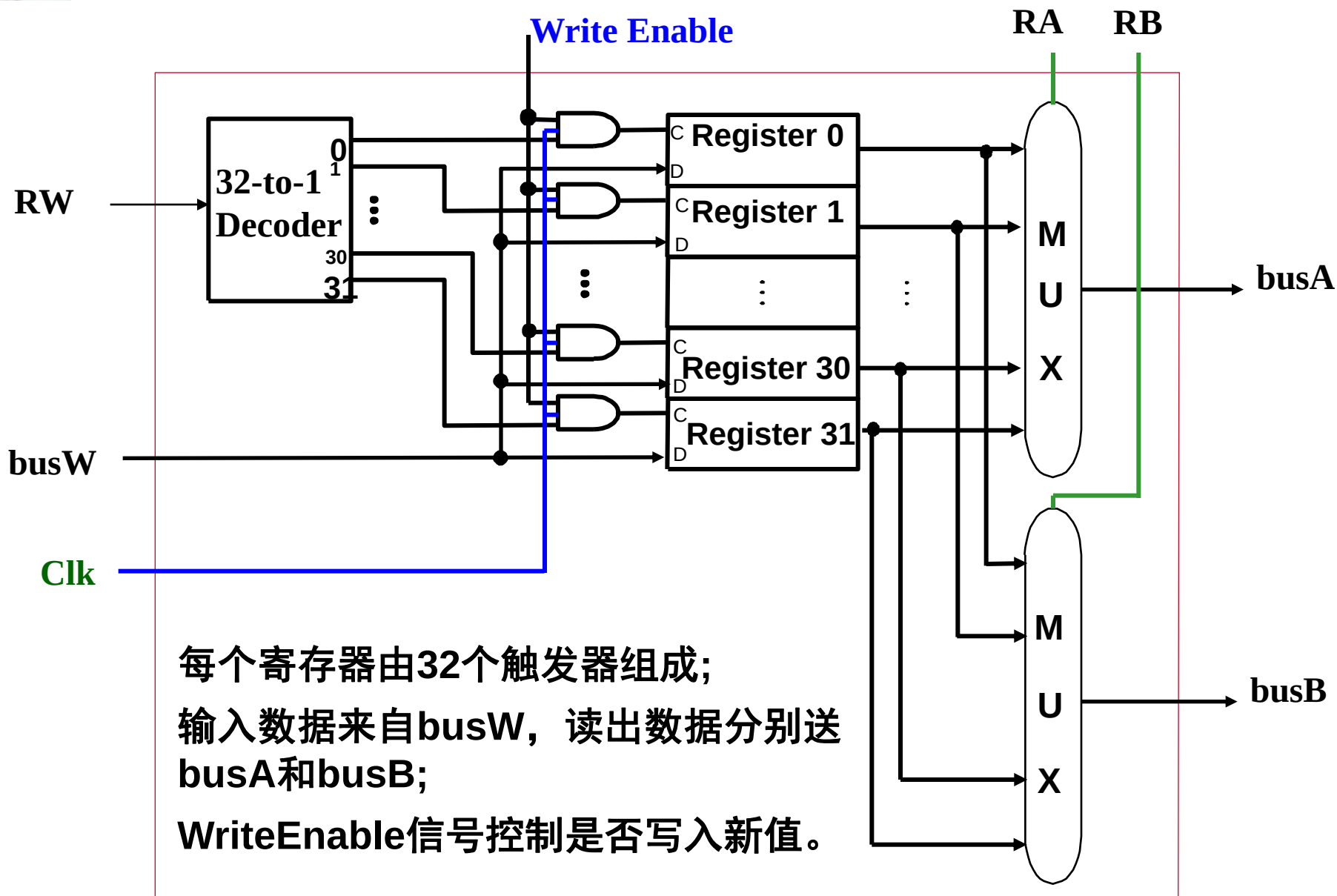


❖ 寄存器组 (Register File)

- 两个读口 (组合逻辑操作): busA和busB分别由RA和RB给出地址。地址RA或RB有效后, 经一个“取数时间 (AccessTime)”, busA和busB有效。
- 一个写口 (时序逻辑操作): 写使能为1的情况下, 时钟边沿到来时, busW传来的值开始被写入RW指定的寄存器中。



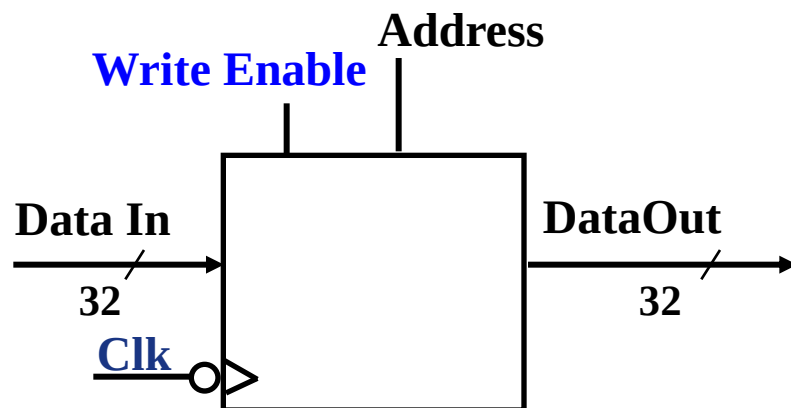
寄存器组的内部结构



存储元件：理想存储器

❖ 理想存储器（idealized memory）

- Data Out: 32位读出数据
- Data In: 32位写入数据
- Address: 读写公用一个32位地址
- 读操作（组合逻辑操作）：地址Address有效后，经一个“取数时间AccessTime”，Data Out上数据有效。
- 写操作（时序逻辑操作）：写使能为1的情况下，时钟Clk边沿到来时，Data In传来的值开始被写入Address指定的存储单元中。



为简化数据通路操作说明，把存储器简化为带时钟信号Clk的理想模型。

数据通路与时序控制

❖ 同步系统(Synchronous system)

- 所有动作有专门时序信号来定时
- 由时序信号规定何时发出什么动作

例如，指令执行过程每一步都有控制信号控制，由定时信号确定控制信号何时发出、作用时间多长

❖ 什么是时序信号？

- 同步系统用于同步控制的定时信号，如时钟信号

❖ 什么叫指令周期？

- 取并执行一条指令的时间
- 每条指令的指令周期肯定一样吗？

❖ 早期计算机的三级时序系统

- 机器周期 - 节拍 - 脉冲
- 指令周期可分为取指令、读操作数、执行并写结果等多个基本工作周期，称为机器周期。
- 机器周期有取指令、存储器读、存储器写、中断响应等不同类型

多级时序系统

1. 机器周期(CPU周期)

(1) 机器周期的概念

所有指令执行过程中的一个基准时间

(2) 确定机器周期需考虑的因素

每条指令的执行 步骤

每一步骤 所需的 时间

(3) 基准时间的确定

- 以完成 最复杂 指令功能的时间 为准
- 以 访问一次存储器 的时间 为基准

若指令字长 = 存储字长 取指周期 = 机器周期

2. 时钟周期（节拍、状态）

- 一个机器周期内可完成若干个微操作
- 每个微操作需一定的时间，以时钟信号来控制产生每一个微操作命令
- 时钟信号控制节拍发生器，产生节拍，每个节拍宽度对应一个时钟周期
- 将一个机器周期分成若干个时间相等的时间段（节拍、状态、时钟周期）
- 时钟周期是控制计算机操作的最小单位时间
- 用时钟周期控制产生一个或几个微操作命令



多级时序系统

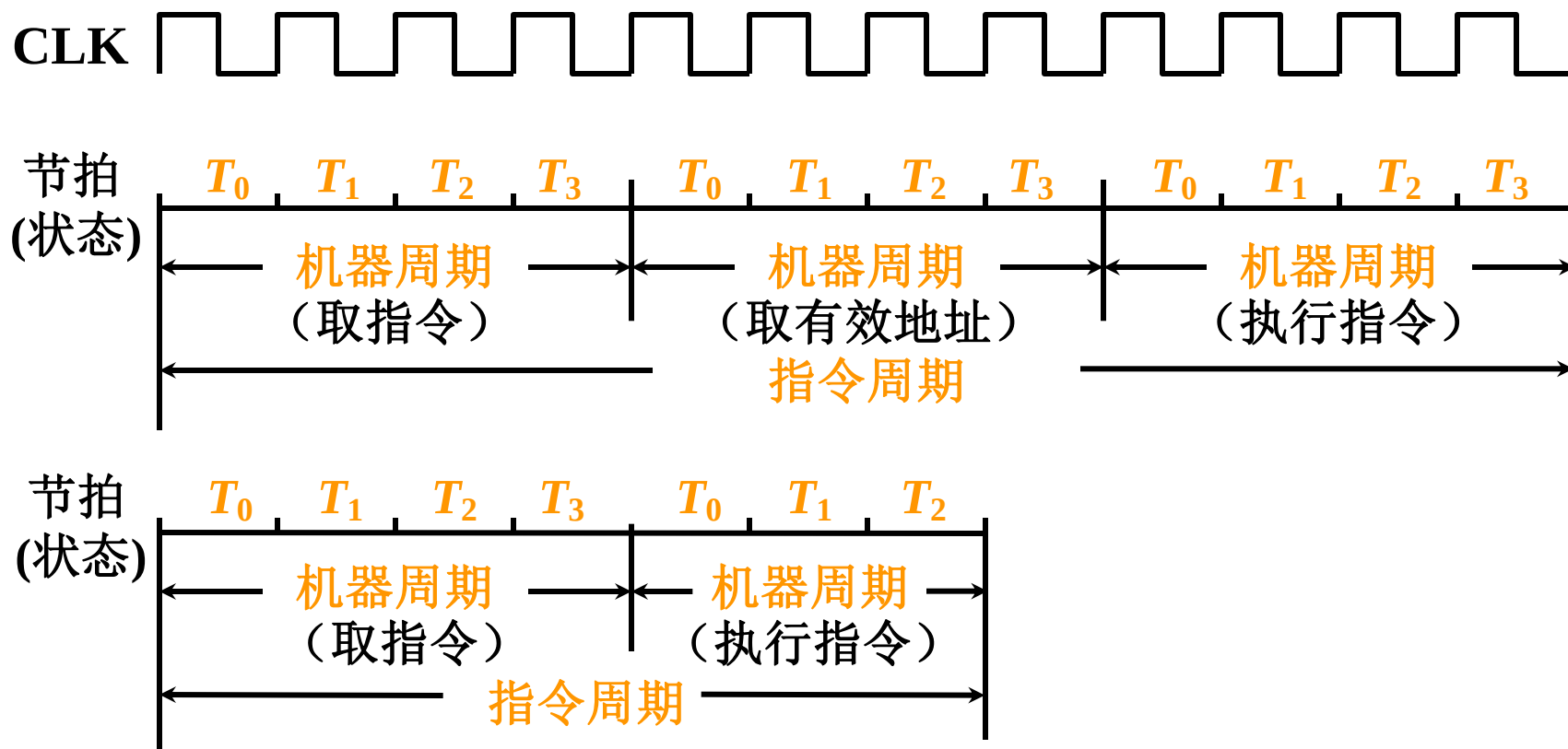
- 指令周期是从取指令、分析指令到执行完该指令所需的时间。
- 不同的指令，其指令周期长短可以不同。
- 在时序系统中通常不为指令周期设置时间标志信号，因而也不将其作为时序的一级

多级时序系统

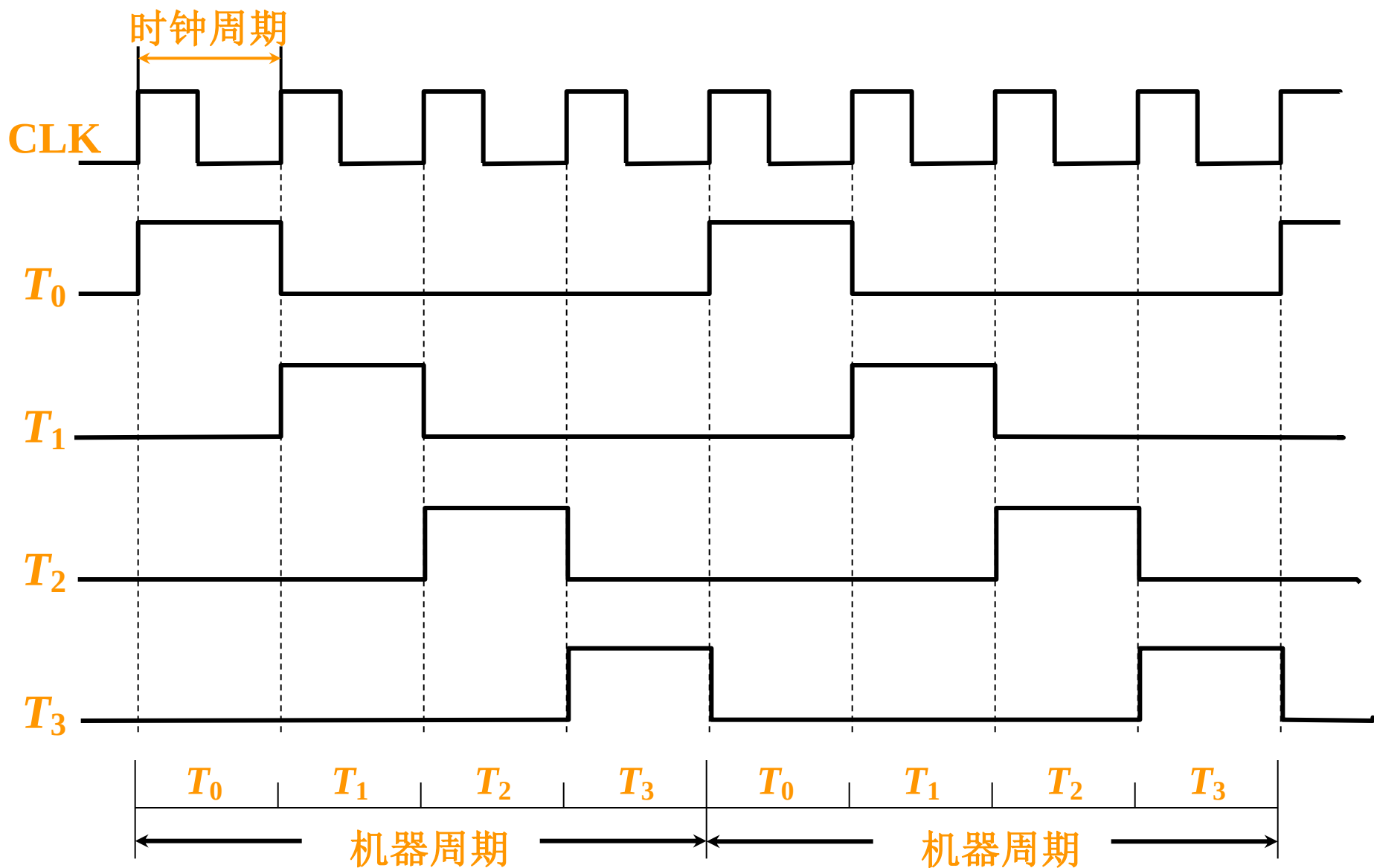
机器周期、节拍（状态）组成多级时序系统

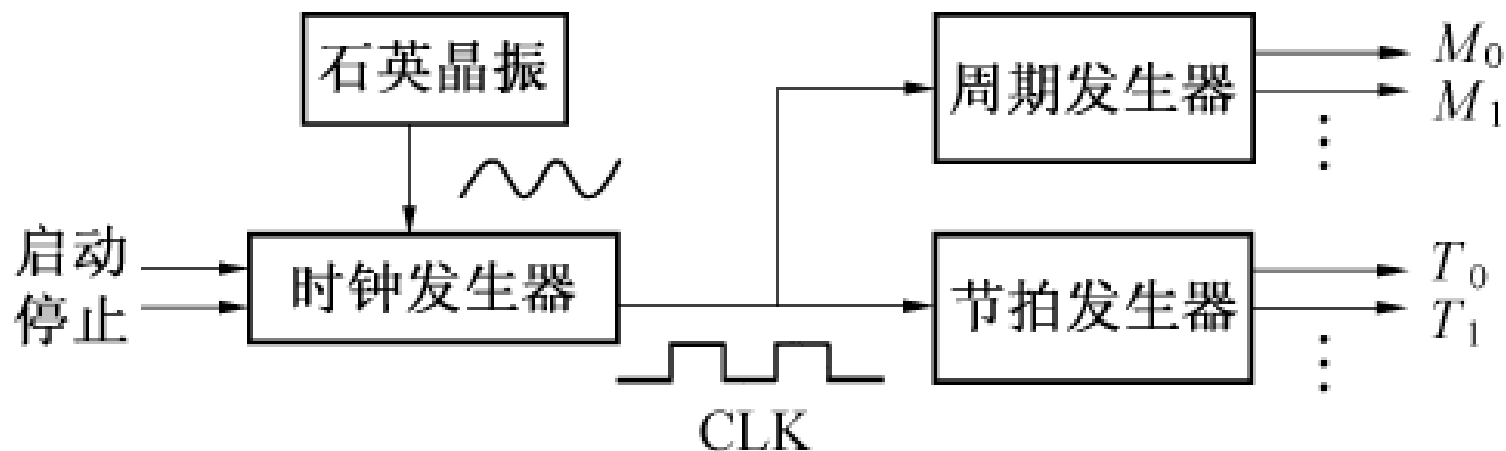
一个指令周期包含若干个机器周期

一个机器周期包含若干个时钟周期

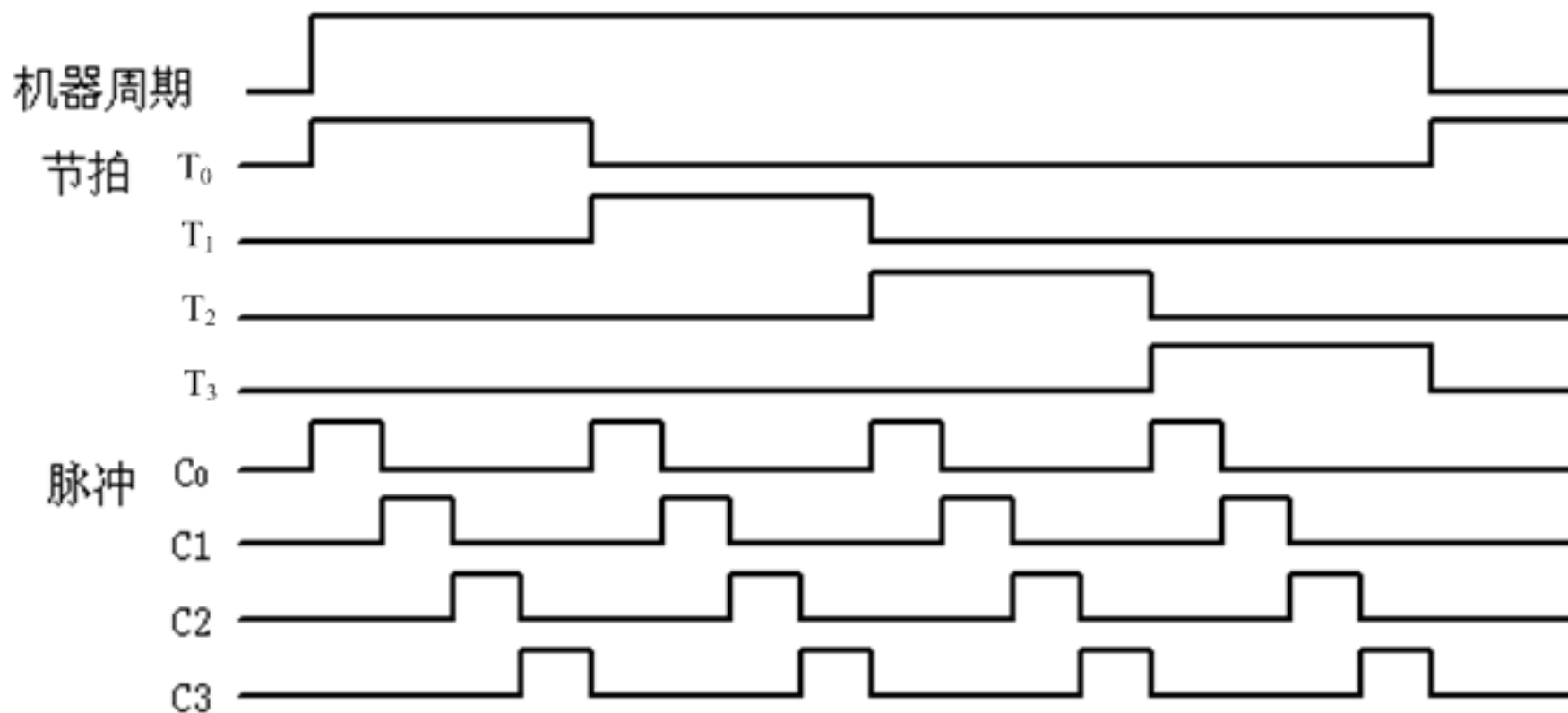


时钟周期（节拍、状态）

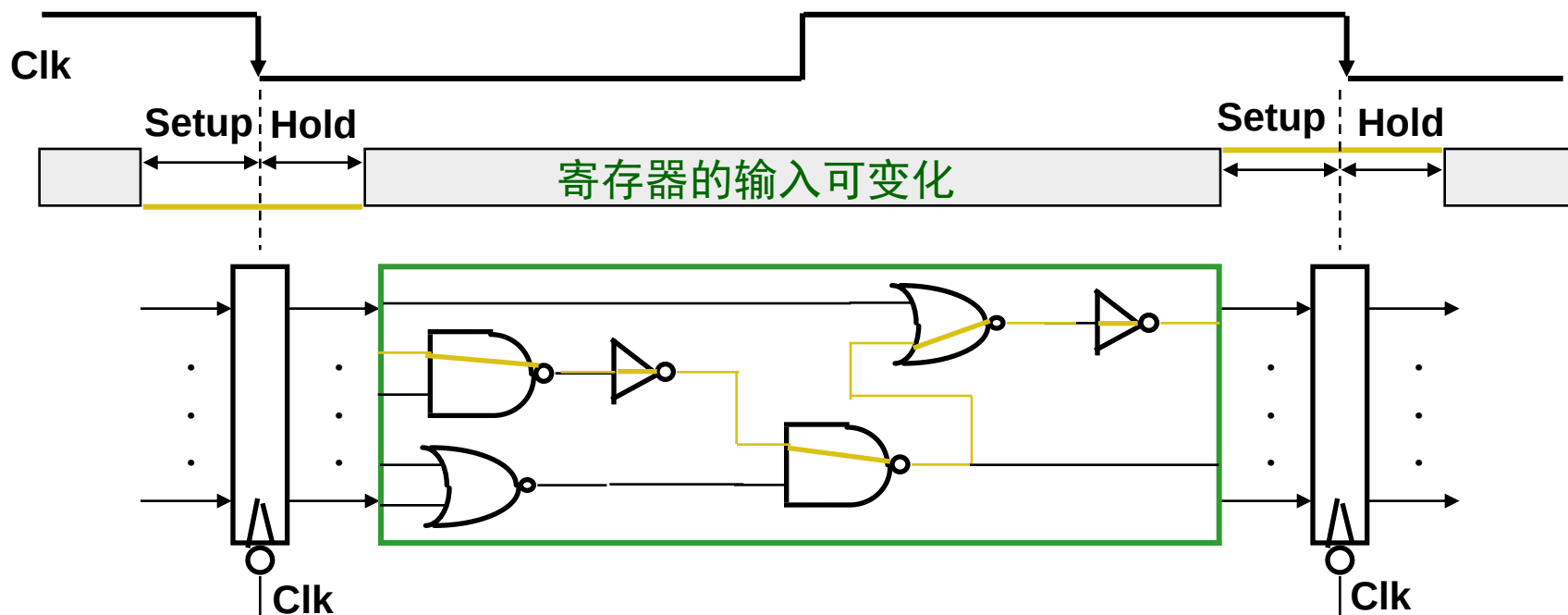




- ❖ 主振电路中的石英晶体振荡器产生一系列正弦信号，经时钟发生器整形分频得到时钟脉冲信号，成为机器的主频脉冲。主频脉冲再经过周期发生器和节拍发生器产生周期电位和节拍电位。



现代计算机已不再采用三级时序系统，机器周期的概念已逐渐消失。整个数据通路中的定时信号就是时钟，一个时钟周期就是一个节拍。



数据通路由 “... + 状态元件 + 操作元件(组合电路) + 状态元件 + ...” 组成

只有状态元件能存储信息，所有操作元件都须从状态单元接收输入，并将输出写入状态单元中。其输入为前一时钟生成的数据，输出为当前时钟所用的数据

- ❖ 假定采用下降沿触发（负跳变）方式（也可以是上升沿方式）
 - 所有状态单元在下降沿写入信息，经过Latch Prop (clk-to-Q) 后输出有效
 - $\text{Cycle Time} = \text{Latch Prop} + \text{Longest Delay Path} + \text{Setup} + \text{Clock Skew(最大偏移)}$
- ❖ 约束条件： $(\text{Latch Prop} + \text{Shortest Delay Path} - \text{Clock Skew}) > \text{Hold Time}$

机器速度与机器主频的关系

- 机器的 **主频 f** 越快 机器的 **速度也越快**
- 在机器周期所含时钟周期数 **相同** 的前提下，
两机 **平均指令执行速度之比** 等于 **两机主频之比**

$$\frac{\text{MIPS}_1}{\text{MIPS}_2} = \frac{f_1}{f_2}$$

- **机器速度** 不仅与 **主频有关**，还与机器周期中所含**时钟周期**（主频的倒数）**数** 以及指令周期中所含的 **机器周期数有关**
- 机器运行速度还与**字长**和**计算机体系结构**有关。
字长越长，单位时间内完成的数据运算就越多，运算速度越快
体系结构：存储器采用分级结构，处理器采用流水结构、多机结构等

8.4 中断系统

一、中断和异常的基本概念

程序执行过程中，CPU会遇到一些特殊情况，使正在执行的程序被“中断”，此时，CPU中止原来正在执行的程序，转到处理异常情况或特殊事件的程序去执行，执行后再返回到原被中止的程序继续执行。

1. 使程序执行被“中断”的事件有两类

- 内部“异常”：在CPU内部发生的意外事件或特殊事件
- 外部“中断”：在CPU外部发生的特殊事件，通过“中断请求”信号向CPU请求处理。

内部“异常”

❖ 按发生原因分为**硬故障中断**和**程序性中断**两类

■ **硬故障中断**：如电源掉电、硬件线路故障等

■ **程序性中断**：执行某条指令时发生的“例外(Exception)”，如溢出、缺页、越界、越权、非法指令、除数为0、堆栈溢出、访问超时、断点、单步、系统调用

❖ 按处理方式分为**故障(fault)**、**自陷(Trap)**和**终止(Abort)**三类

■ **故障**：执行指令引起的异常事件，如溢出、缺页、堆栈溢出、访问超时等

■ **自陷**：预先安排的事件，如单步跟踪、系统调用(执行访管指令)等 (**自愿中断**)

■ **终止**：硬故障事件，机器将“终止”，调出中断服务程序来重启操作系统

思考：哪些故障补救后可继续执行，哪些只好终止当前进程？

- 缺页等：补救后可继续，回到发生故障的指令重新执行
- 溢出、除数为0、非法操作、内存保护错等：终止当前进程

思考：自陷处理完成后回到哪条指令执行

- 回到下一条指令

外部 “中断”

- ❖ 外部中断：在CPU外部发生的特殊事件，通过 “中断请求” 信号向CPU请求处理
 - 如实时钟、控制台、打印机缺纸、外设准备好、采样计时到、DMA传输结束等
 - 中断是一种I/O方式，所以有关程序中断的理论在第5章

举例——8086/8088中断系统

统称为“中断”：内中断（内部异常）和外中断（外部中断）

◦ 内中断：**CPU**自己产生而不通过中断请求线请求，皆为不可屏蔽中断。

- 指令引起异常：**CPU**执行预置的指令后在特定的情况下发生的异常。

INTO 溢出：执行算术指令后，若发生溢出，则产生**类型4**中断。

INT n 用户定义：指令的第二字节给出一个类型号($n=0\sim 255$)。

其中 $n=3$ (**INT 3**)时为断点设置，该指令执行后，自动产生**类型3**中断。

- 处理器检测异常：**CPU**执行指令时产生的异常，如：除法错、无效操作码、缺页、单步跟踪调试等。如：

除法错：除数为0或商溢出，则产生**类型0**中断。

单步跟踪：当自陷位 $TF=1$ 且处在开中断状态(即 $IF=1$)时，每条指令执行完就自动产生**类型1**中断。

◦ 外中断：通过中断请求线**INTR**和**NMI**来实现。

- INTR**：可屏蔽中断 (外设中断源引起的中断)。
- NMI**：不可屏蔽中断 (重要或紧急的硬件故障)，属于**类型2**中断。

所有事件都被分配一个“中断类型号”

每个中断都有相应的“中断服务程序”

可根据中断类型号找到中断服务程序的入口地址

按中断服务程序入口地址的获取方式分类

- (1) **向量中断**：外部设备在提出中断请求的同时，通过**硬件**自动形成**中断服务程序入口地址**。**CPU**响应中断后，将根据向量地址直接转入相应中断服务程序。这种具有产生向量地址的中断称为向量中断。
- (2) **非向量中断**：非向量中断在产生中断的同时不能直接提供中断服务程序入口地址，而只产生一个**中断查询程序的入口地址**。需要通过中断查询程序确定中断源和中断服务程序的入口地址。

2. 中断系统需解决的问题

- (1) 各中断源 如何 向 CPU 提出请求？
- (2) 各中断源 同时 提出 请求 怎么办？
- (3) CPU 什么 条件、什么 时间、以什么 方式 响应中断？
- (4) 如何 保护现场？
- (5) 如何 寻找入口地址？
- (6) 如何 恢复现场，如何 返回？
- (7) 处理中断的过程中又 出现新的中断 怎么办？

硬件 + 软件

二、中断请求标记和中断判优逻辑

8.4

1. 中断请求标记 INTR

一个请求源 一个 INTR 中断请求触发器

多个 INTR 组成 中断请求标记寄存器

1	2	3	4	5			n
掉电	过热	主存读写校验错	阶上溢	非法除法		键盘输入	打印机输出

INTR 分散 在各个中断源的 接口电路中

INTR 集中在 CPU 的中断系统 内

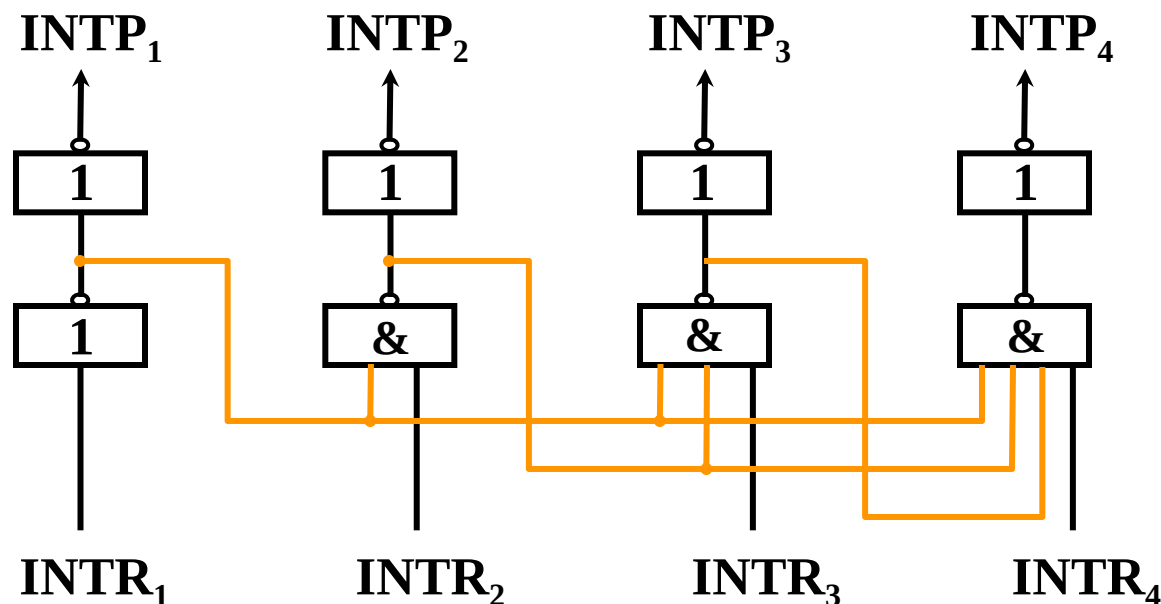
2. 中断判优逻辑

(1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中 链式排队器

参见 第五章

② 集中 在 CPU 内

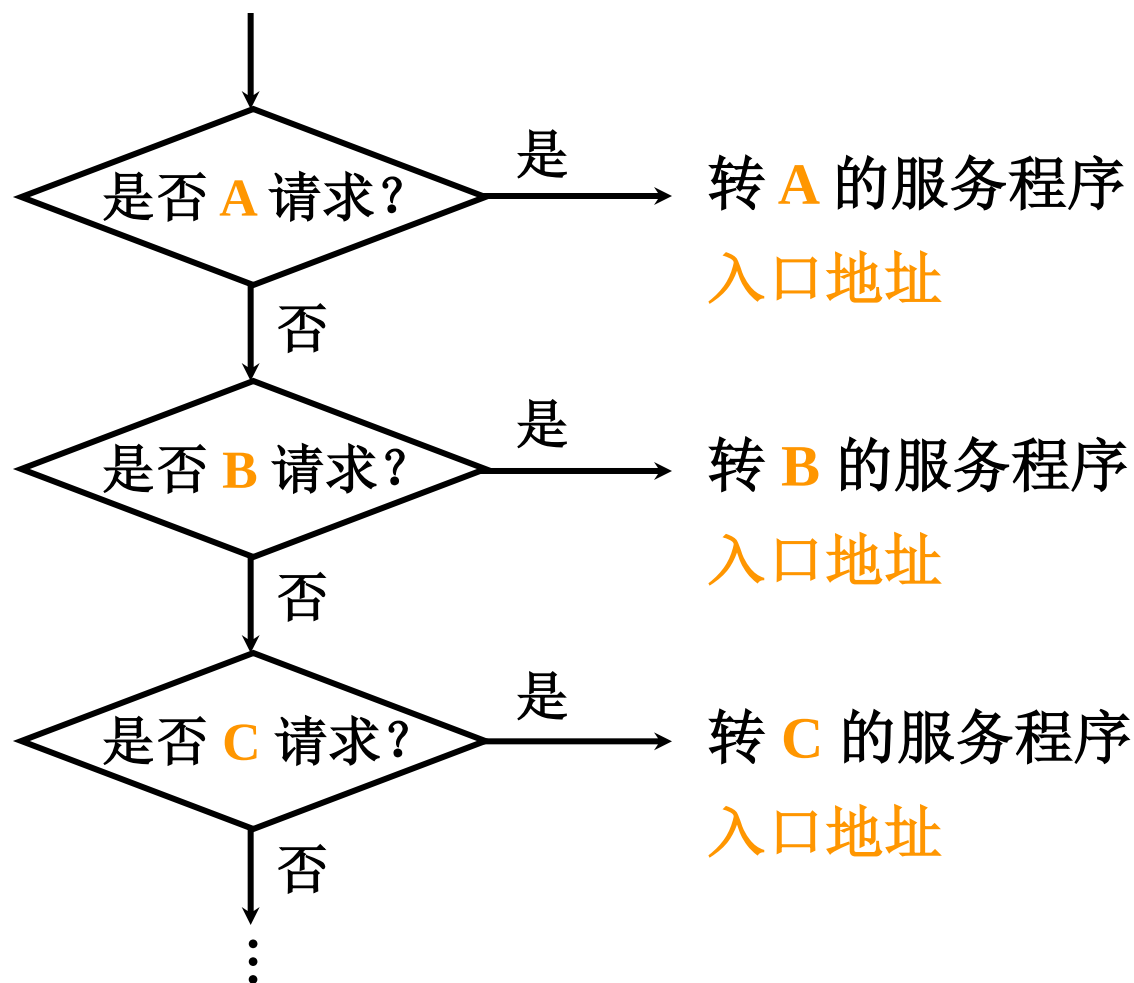


$INTR_1$ 、 $INTR_2$ 、 $INTR_3$ 、 $INTR_4$ 优先级按降序排列

(2) 软件实现（程序查询）

8.4

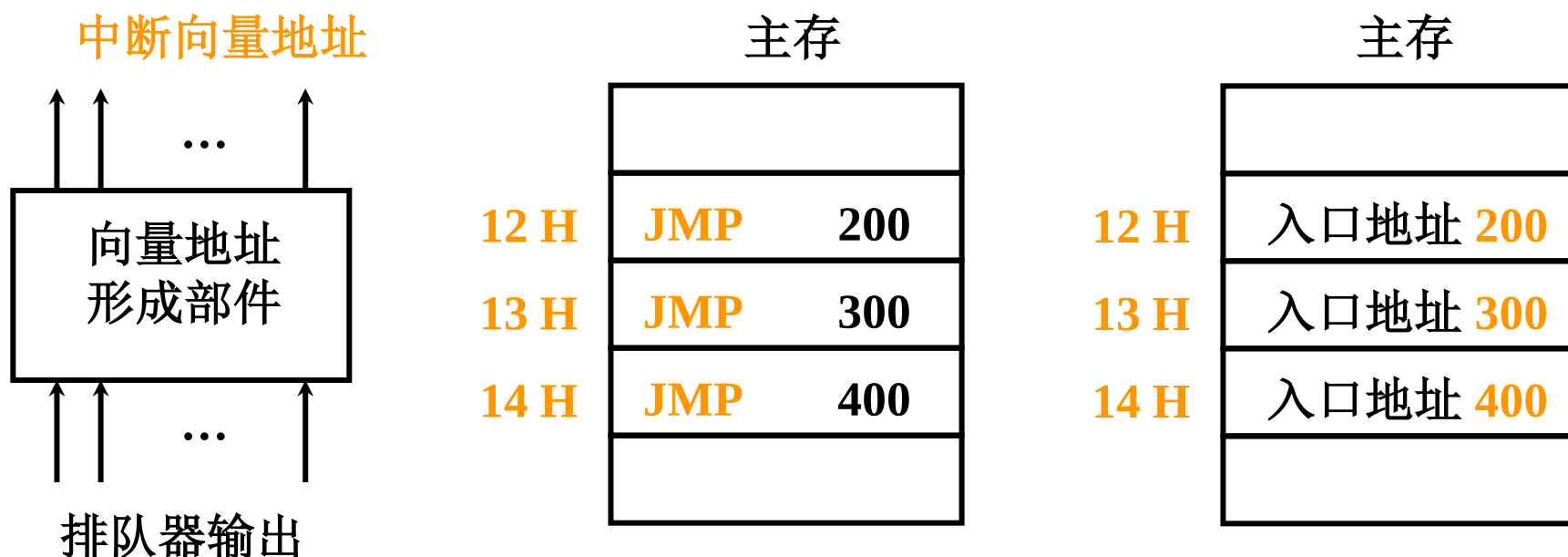
A、B、C 优先级按 降序 排列



三、中断服务程序入口地址的寻找

8.4

1. 硬件向量法



向量地址 12H、13H、14H

入口地址 200、300、400

8086/8088的中断向量表

中断向量表也称中断入口地址表（或异常表），位于0000H～03FFH。共256组，每组占四个字节 CS:IP。向量地址=中断类型号 × 4

例1：除法错的中断类型号为0，故其向量地址为：0x4=0

除法错
单步

例2：NMI的中断类型号为2，故其向量地址为：2x4=8

NMI

⋮

CS:IP	000～003H
CS:IP	004～007H
CS:IP	008～00BH
⋮	
CS:IP	
CS:IP	3FC～3FFH

- 中断向量表（异常表）中每一项是对应中断服务程序的入口地址。被称为中断向量(Interrupt Vector)
- 中断向量表的起始地址存放在一个异常表基址寄存器中。

2. 软件查询法

八个中断源 1, 2, ... 8 按 降序 排列

中断识别程序（入口地址 **M**）

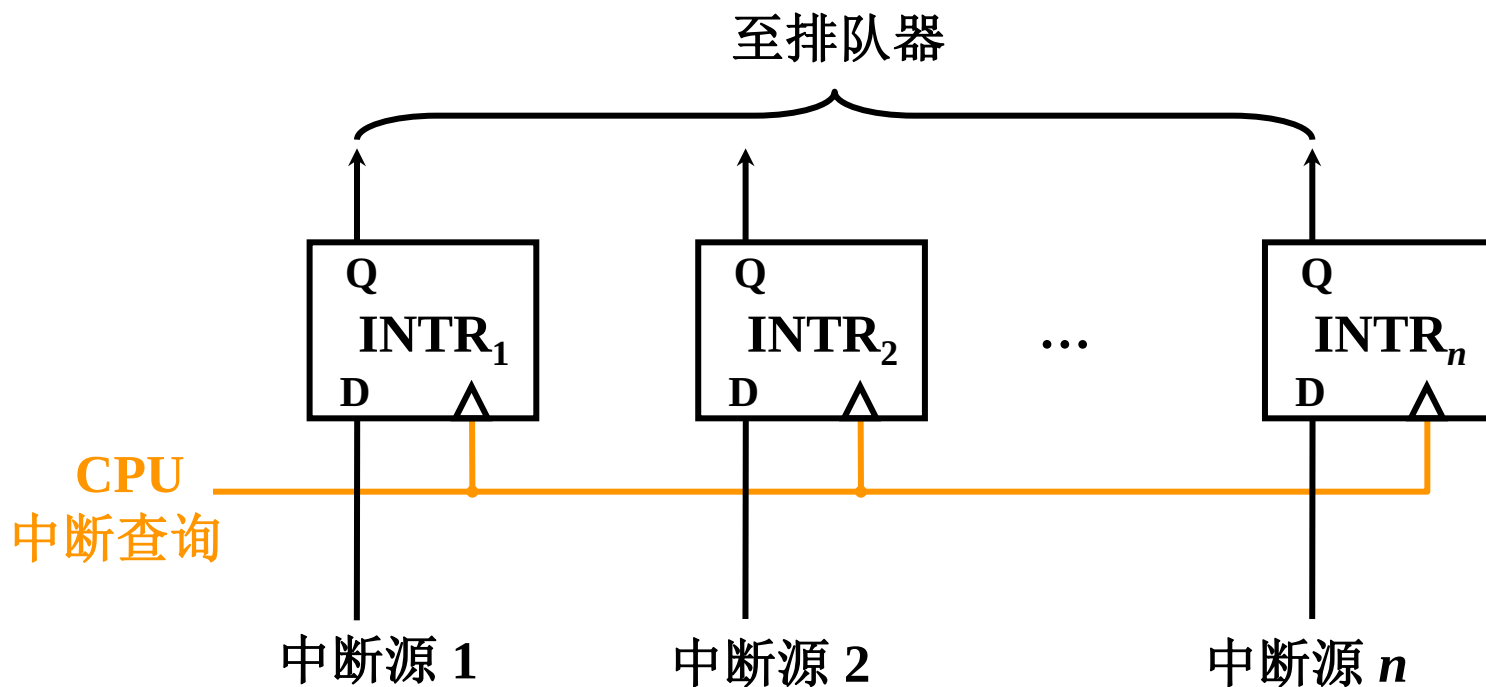
地 址	指 令	说 明
M	SKP DZ 1 [#]	1 [#] D = 0 跳（D为完成触发器）
	JMP 1 [#] SR	1 [#] D = 1 转1 [#] 服务程序
	SKP DZ 2 [#]	2 [#] D = 0 跳
	JMP 2 [#] SR	2 [#] D = 1 转2 [#] 服务程序
	⋮	
	SKP DZ 8 [#]	8 [#] D = 0 跳
	JMP 8 [#] SR	8 [#] D = 1 转8 [#] 服务程序

1. 响应中断的 条件

允许中断触发器 **EINT = 1** (可被开中断指令置1也可被关中断指令置0)

2. 响应中断的 时间

指令执行周期结束时刻由CPU 发查询信号



3. 中断隐指令

8.4

(1) 保护程序断点

断点存于 **特定地址**（0 号地址）内 断点 **进栈**

(2) 寻找服务程序入口地址

向量地址 $\longrightarrow$ **PC** （硬件向量法）

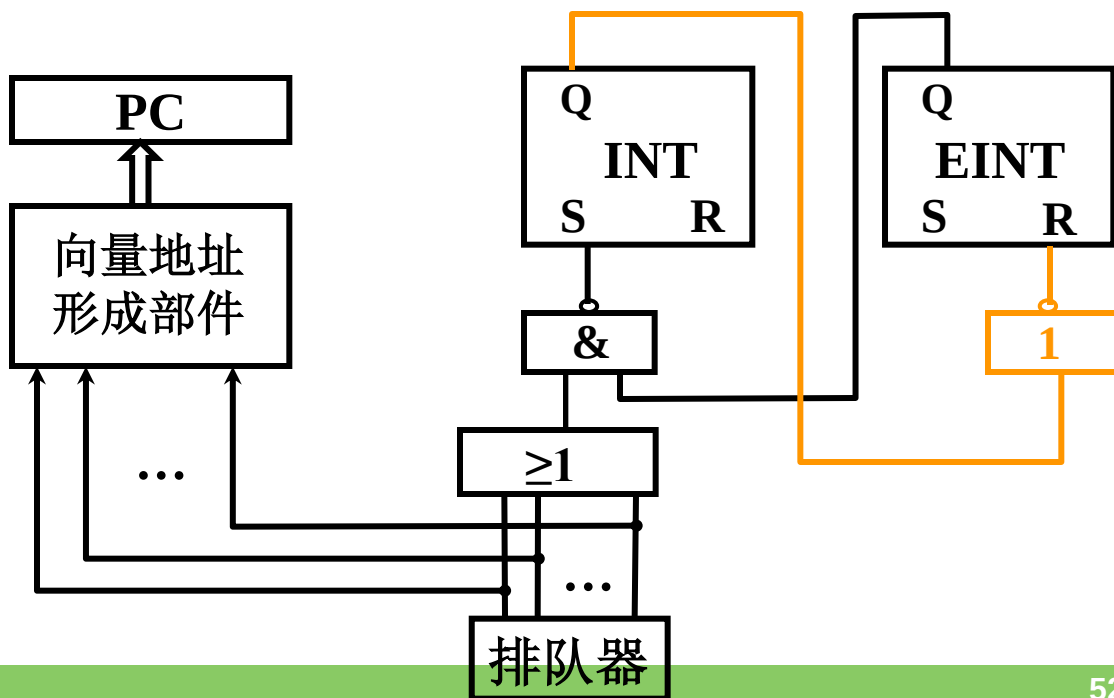
中断识别程序 入口地址 **M** $\longrightarrow$ **PC** （软件查询法）

(3) 硬件 **关中断**

INT 中断标记

EINT 允许中断

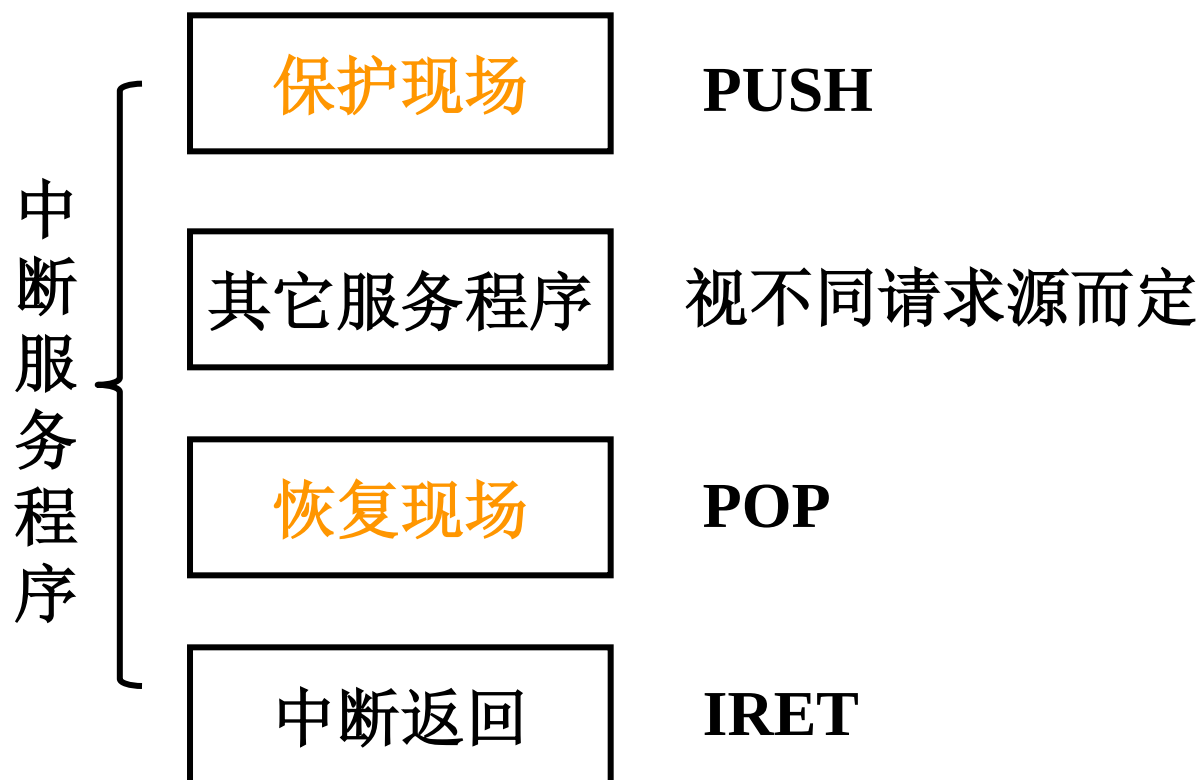
R - S 触发器



五、保护现场和恢复现场

8.4

1. 保护现场 { 断点 中断隐指令 完成
寄存器 内容 中断服务程序 完成
2. 恢复现场 中断服务程序 完成



处理器中的异常处理机制

检测到异常时，处理器必须进行以下基本处理：

① 关中断：使处理器处于“禁止中断”状态，以防止新异常(或中断)破坏断点和现场

② 保护断点和程序状态：将断点和程序状态保存到堆栈或特殊寄存器中

PC→堆栈 或 EPC（专门存放断点的寄存器）

PSWR →堆栈 或 EPSWR（专门保存程序状态的寄存器）

（PSW（Program Status Word）：程序状态字，包括条件码、中断码、状态位等

PSWR（PSW寄存器）：用于存放程序状态字的寄存器。如，X86的FLAGS）

③ 识别异常事件：有两种不同的方式：软件识别和硬件识别（向量中断方式）

（1）软件识别（MIPS采用）

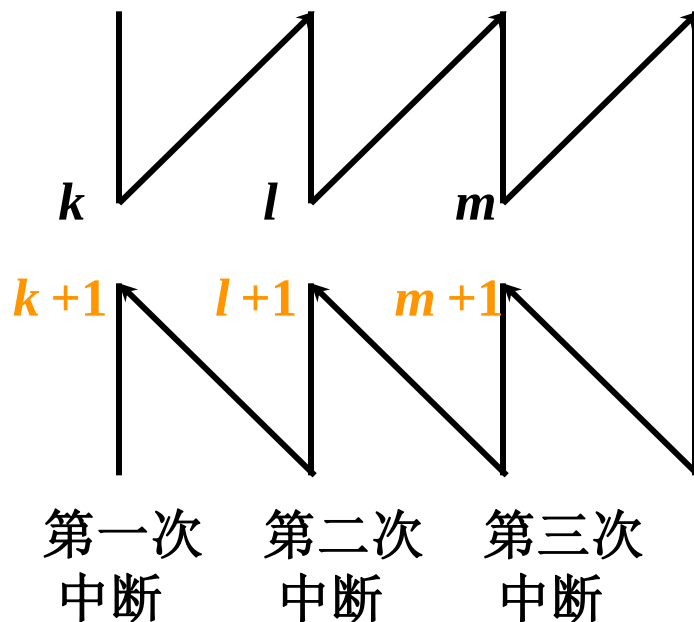
设置一个异常状态寄存器（MIPS中为Cause寄存器），用于记录异常原因。操作系统使用一个统一的异常处理程序，该程序按优先级顺序查询异常状态寄存器，识别出异常事件。

（例如：MIPS中位于内核地址0x8000 0180处有一个专门的异常处理程序，用于检测异常的具体原因，然后转到内核中相应的异常处理程序段中进行具体的处理）

（2）硬件识别（向量中断）（80x86采用）

用专门的硬件查询电路按优先级顺序识别异常，得到“中断类型号”，根据此号，到中断向量表中读取对应的中断服务程序的入口地址。

1. 多重中断的概念



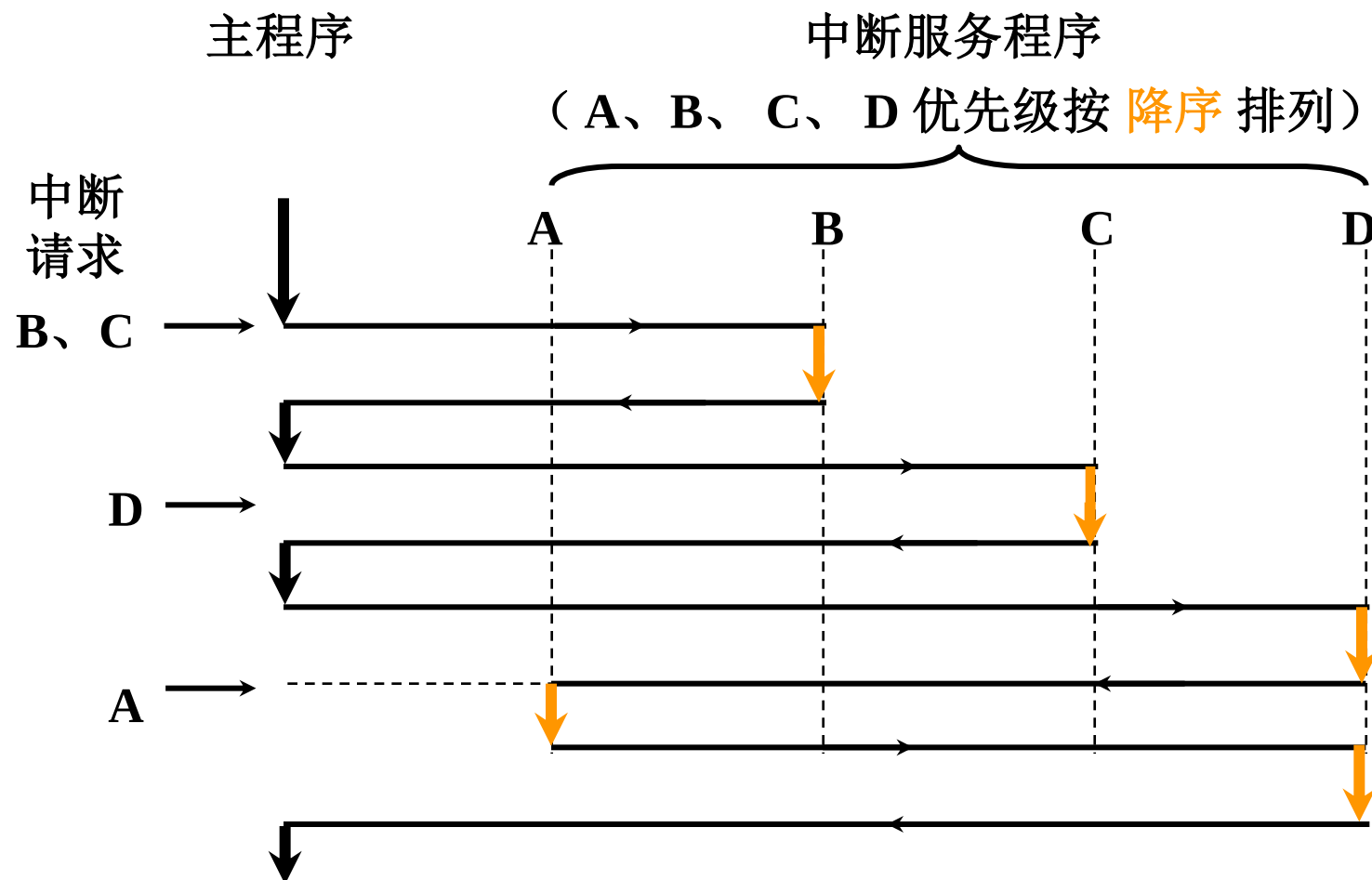
程序断点 $k+1$, $l+1$, $m+1$

2. 实现多重中断的条件

8.4

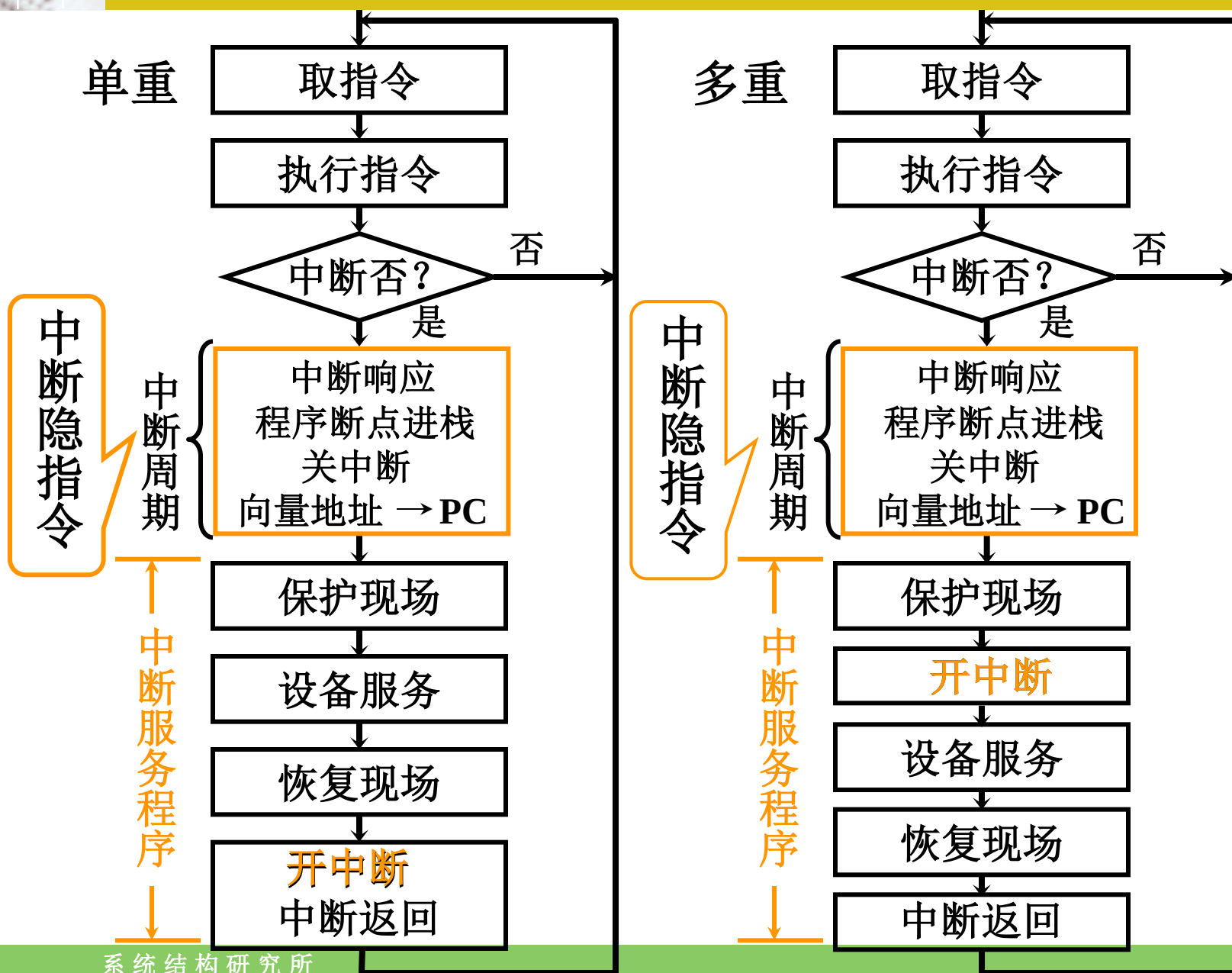
(1) 提前 设置 开中断 指令

(2) 优先级别高 的中断源 有权中断优先级别低 的中断源



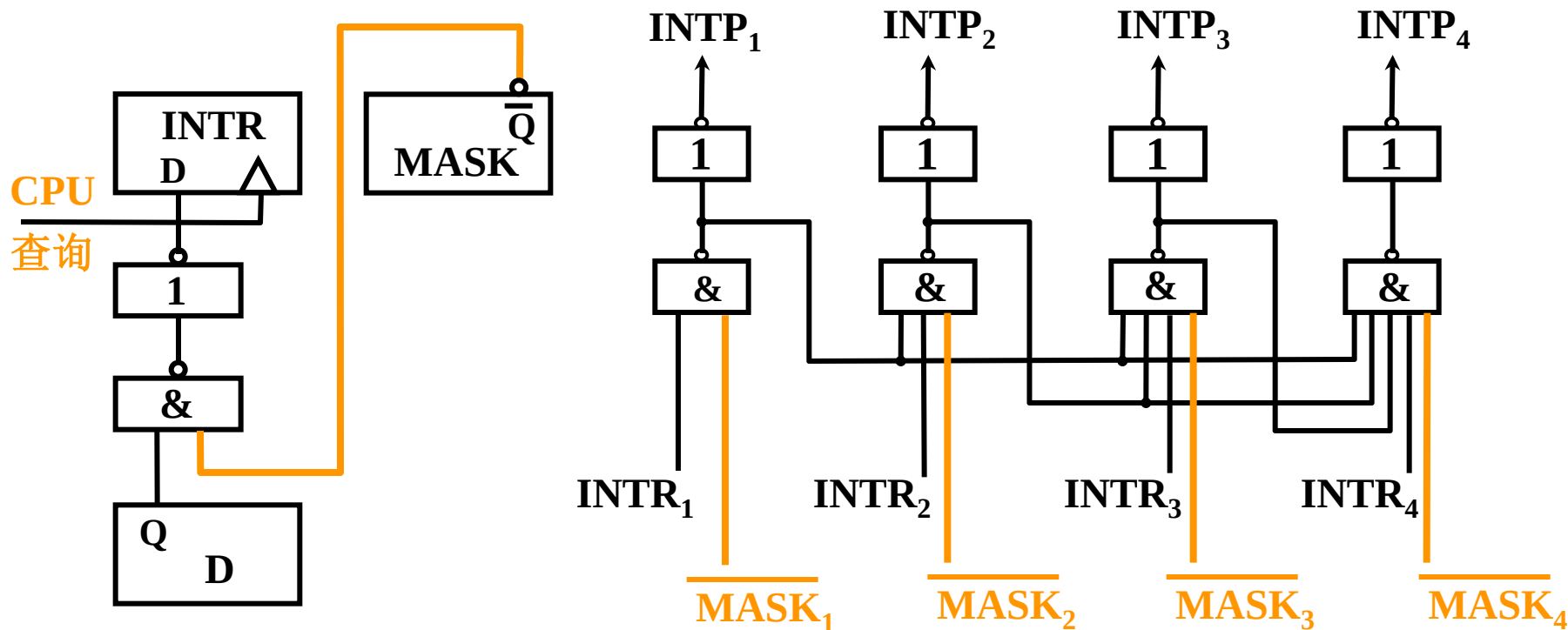
单重中断和多重中断的服务程序流程

5.5



3. 屏蔽技术

(1) 屏蔽触发器的作用



$MASK = 0$ (未屏蔽)

INTR 能被置“1”

$MASK_i = 1$ (屏蔽)

$INTP_i = 0$ (不能被排队选中)

(2) 屏蔽字

8.4

16个中断源 1, 2, 3, ..., 16 按 降序 排列

优先级	屏蔽字															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
5	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
⋮	⋮															
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

(3) 屏蔽技术可改变处理优先等级

8.4

响应优先级 不可改变

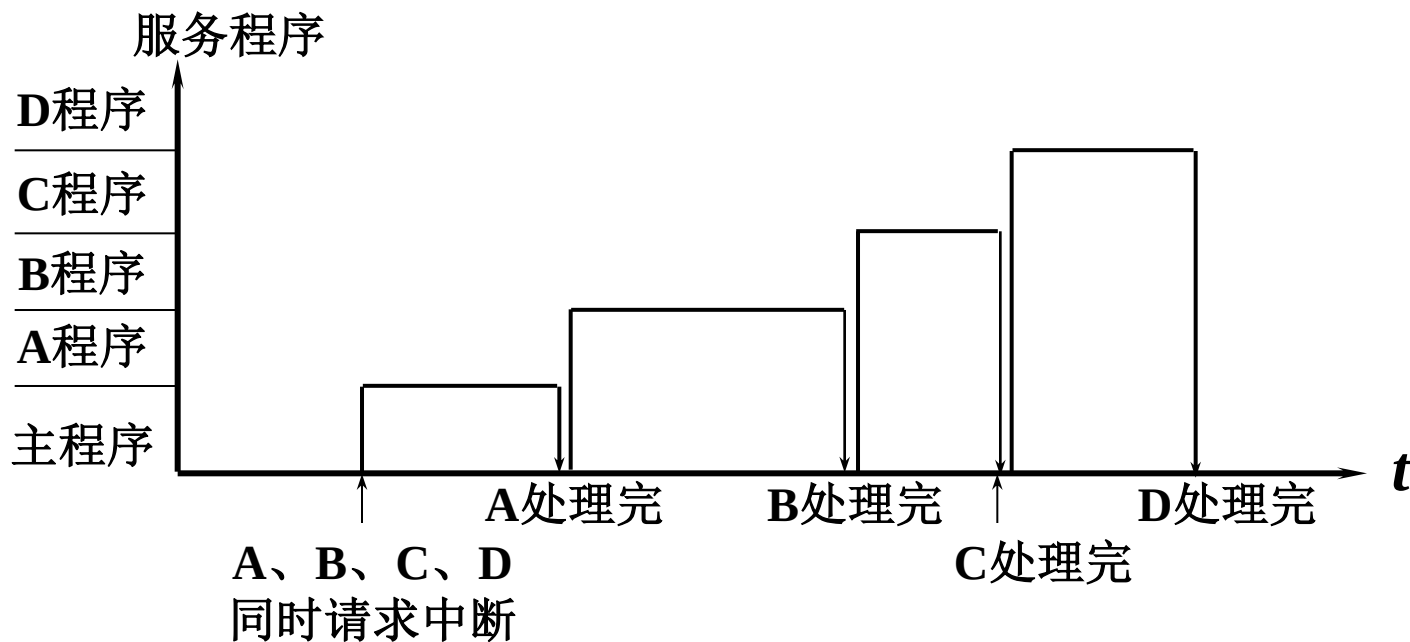
处理优先级 可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 **A→B→C→D** 降序排列

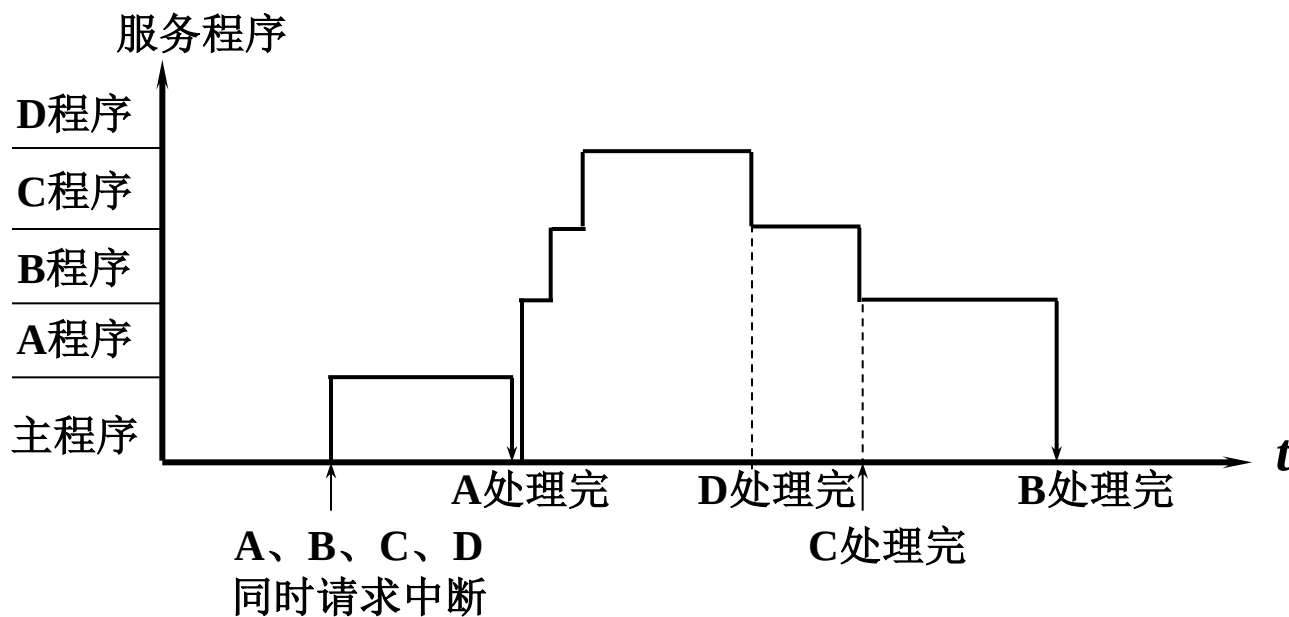
处理优先级 **A→D→C→B** 降序排列

(3) 屏蔽技术可改变处理优先等级



CPU 执行程序轨迹（原屏蔽字）

(3) 屏蔽技术可改变处理优先等级



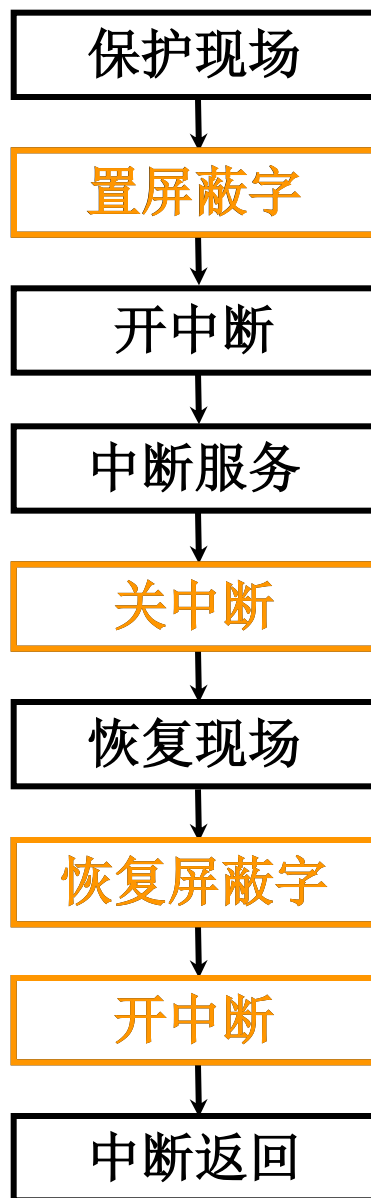
CPU 执行程序轨迹（新屏蔽字）

(4) 屏蔽技术的其他作用

可以 **人为地屏蔽** 某个中断源的请求
便于程序控制

(5) 新屏蔽字的设置

8.4



4. 多重中断的断点保护

8.4

(1) 断点进栈 中断隐指令 完成

(2) 断点存入 “0” 地址 中断隐指令 完成

中断周期 0 $\rightarrow$ MAR

命令存储器写

PC $\rightarrow$ MDR 断点 $\rightarrow$ MDR

(MDR) $\rightarrow$ 存入存储器

三次中断，三个断点都存入 “0” 地址

？ 如何保证断点不丢失？

例 题

1. 某机有五个中断源，按中断响应的优先顺序由高到低为L0,L1,L2,L3,L4，现要求优先顺序改为L4,L2,L3,L0,L1，写出各中断源的屏蔽字。

中断源	屏蔽字				
	0	1	2	3	4
L0					
L1					
L2					
L3					
L4					

中断源	屏蔽字				
	0	1	2	3	4
L0	1	1	0	0	0
L1	0	1	0	0	0
L2	1	1	1	1	0
L3	1	1	0	1	0
L4	1	1	1	1	1

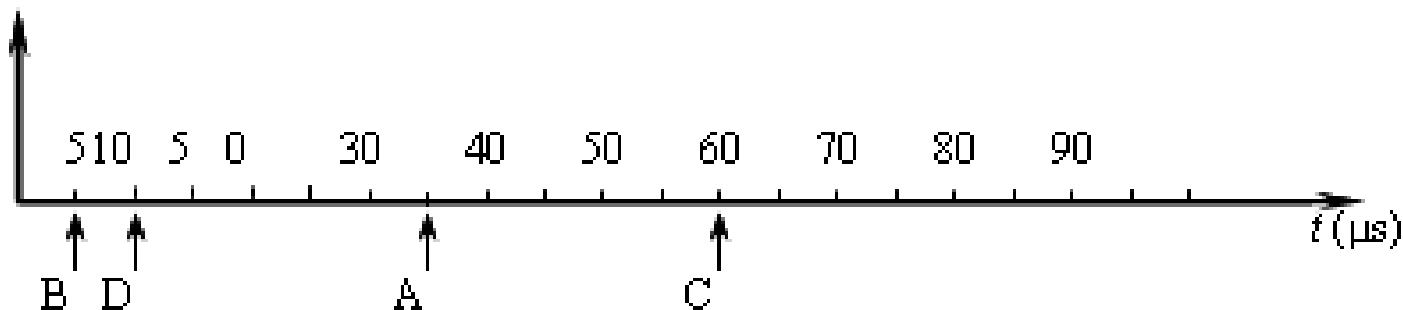
例 题

2. 设某机有四个中断源A、B、C、D，其硬件排队优先次序为 $A > B > C > D$ ，现要求将中断处理次序改为 $D > A > C > B$ 。

(1) 写出每个中断源对应的屏蔽字。

(2) 按下图时间轴给出的四个中断源的请求时刻，画出CPU执行程序的轨迹。设每个中断源的中断服务程序时间均为 $20\mu\text{s}$ 。

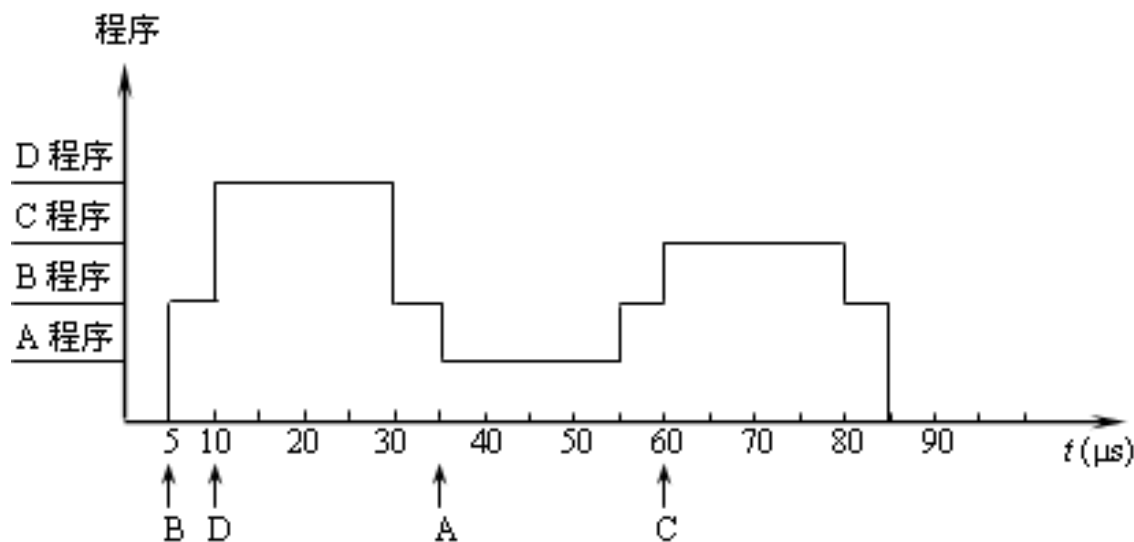
程序



答：（1）在中断处理次序
改为 $D > A > C > B$ 后，
每个中断源新的屏蔽字如
表所示。

中断源	屏蔽字			
	A	B	C	D
A	1	1	1	0
B	0	1	0	0
C	0	1	1	0
D	1	1	1	1

（2）根据新的处理次序，
CPU执行程序的轨迹如图所示



单周期计算机的性能

- ❖ 单周期处理器的CPI为多少？ $CPI=1!$
 - 其他条件一定的情况下，CPI越小，则性能越好！
 - $CPI=1$ ，不是很好吗？
- ❖ 单周期处理器的性能会不会很好？为什么？
 - 计算机的性能除CPI外，还取决于时钟周期的宽度
 - 单周期处理器的时钟宽度为最复杂指令的执行时间
 - 很多指令可以在更短的时间内完成

Computer Organization



Thank You !

系统结构研究所